澳門理工大學
Universidade Politécnica de Macau
Macao Polytechnic University

Introduction to Data Analytics

Jie Zhang
Faculty of Applied Sciences, Macao Polytechnic University
Email: jpeter.zhang@mpu.edu.mo
Personal Profile: easy-shu.github.io



Data analytics

Data analytics is a valuable tool for businesses aiming to increase revenue, improve products, and retain customers. According to research by global management consulting firm McKinsey & Company, companies that use data analytics are 23 times more likely to outperform competitors in terms of new customer acquisition than non-data-driven companies. They were also nine times more likely to surpass them in measures of customer loyalty and 19 times more likely to achieve above-average profitability.

Data analytics can be broken into four key types:

- Descriptive, which answers the question, "**What happened?**"
- Diagnostic, which answers the question, "**Why did this happen?**"
- Predictive, which answers the question, "**What might happen in the future?**"
- Prescriptive, which answers the question, "**What should we do next?**"

Each type of data analysis can help you reach specific goals and be used in tandem to create a full picture of data that informs your organization's strategy formulation and decision-making.

Source: https://online.hbs.edu/blog/post/types-of-data-analysis



Descriptive Analytics: "What happened?"

Descriptive analytics aims to summarize historical data to uncover trends, patterns, or performance insights.

Key methods and tools include:

- **Statistical Descriptive Methods**: Utilizes measures like mean, median, mode, variance, and standard deviation for summarizing data distributions.
- **Clustering**: Helps in summarizing data segments using algorithms like K-Means and DBSCAN.
- **Dimensionality Reduction**: Simplifies large datasets by reducing input variables while retaining important information for better data understanding and visualization.
- **Time Series Analysis**: Describes trends over time, often utilizing moving averages or seasonal decomposition techniques.



Diagnostic Analytics: "Why did this happen?"

Diagnostic analytics focuses on identifying underlying causes behind data trends or patterns.

Common methods and tools include:

- **Correlation and Regression Analysis**: Determines relationships between variables using techniques like Linear Regression and Logistic Regression.
- **Classification**: Uses labeled data to diagnose categories for new data points with algorithms like Decision Trees, Random Forest, and Support Vector Machines (SVM).
- **Root Cause Analysis (RCA) :** Identifies root causes of specific issues using methods like Ishikawa (Fishbone) diagrams and the 5 Whys technique.
- **Anomaly Detection**: Identifies outliers or unusual data points that may indicate problems using algorithms like Z-score and Isolation Forest.
- **Time Series Decomposition**: Breaks down time series data to identify trend, seasonality, and irregular components.



Predictive Analytics: "What might happen in the future?"

Predictive analytics leverages historical data to make forecasts about future trends or behaviors.

Common methods and algorithms include:

- **Regression Analysis**: Predicts continuous outcomes by identifying relationships between independent and dependent variables with techniques like Multiple Linear Regression and Ridge Regression.
- **Classification and Predictive Modeling**: Categorizes data into predefined classes using algorithms like Random Forest, Gradient Boosting Machines (GBM), and XGBoost.
- **Neural Networks (Deep Learning)**: Utilizes models like Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Convolutional Neural Networks (CNN) for capturing complex relationships.
- **Ensemble Models**: Combines multiple predictive models to enhance accuracy with methods like Bagging and Boosting (e.g., AdaBoost, CatBoost).
- **Time Series Forecasting**: Predicts future values based on historical data using algorithms like ARIMA, Prophet, and Exponential Smoothing.



Prescriptive Analytics: "What should we do next?"

Prescriptive analytics provides recommendations based on data insights to facilitate decision making.

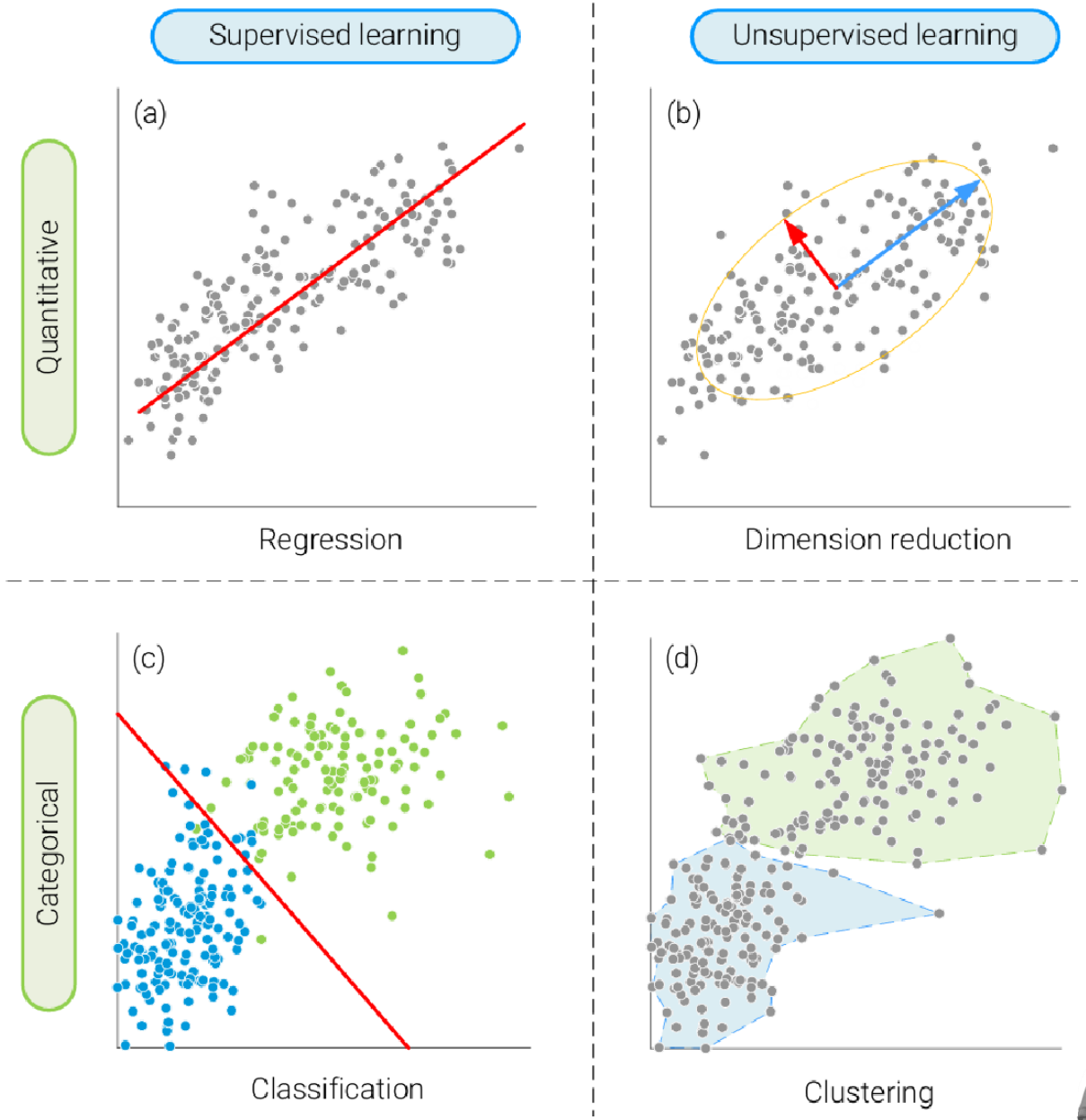
Common methods and tools include:

- **Optimization Models**: Helps in making optimal decisions under constraints using algorithms like Linear Programming (LP), Integer Programming (IP), and Genetic Algorithms.
- **Simulation Modeling**: Uses models to simulate different scenarios and outcomes with methods like Monte Carlo Simulations and Agent-based Models.
- **Reinforcement Learning (RL):** Learns optimal actions through trial and error with models such as Q-learning and Deep Q Networks (DQN).
- **Decision Trees**: Aids in decision-making by providing structured paths of possible outcomes using algorithms like CART and C4.5.
- **Markov Decision Processes (MDP):** Models decision-making in partially random environments, useful for policy optimization and complex system recommendations.



Four Main Machine Learning Methods

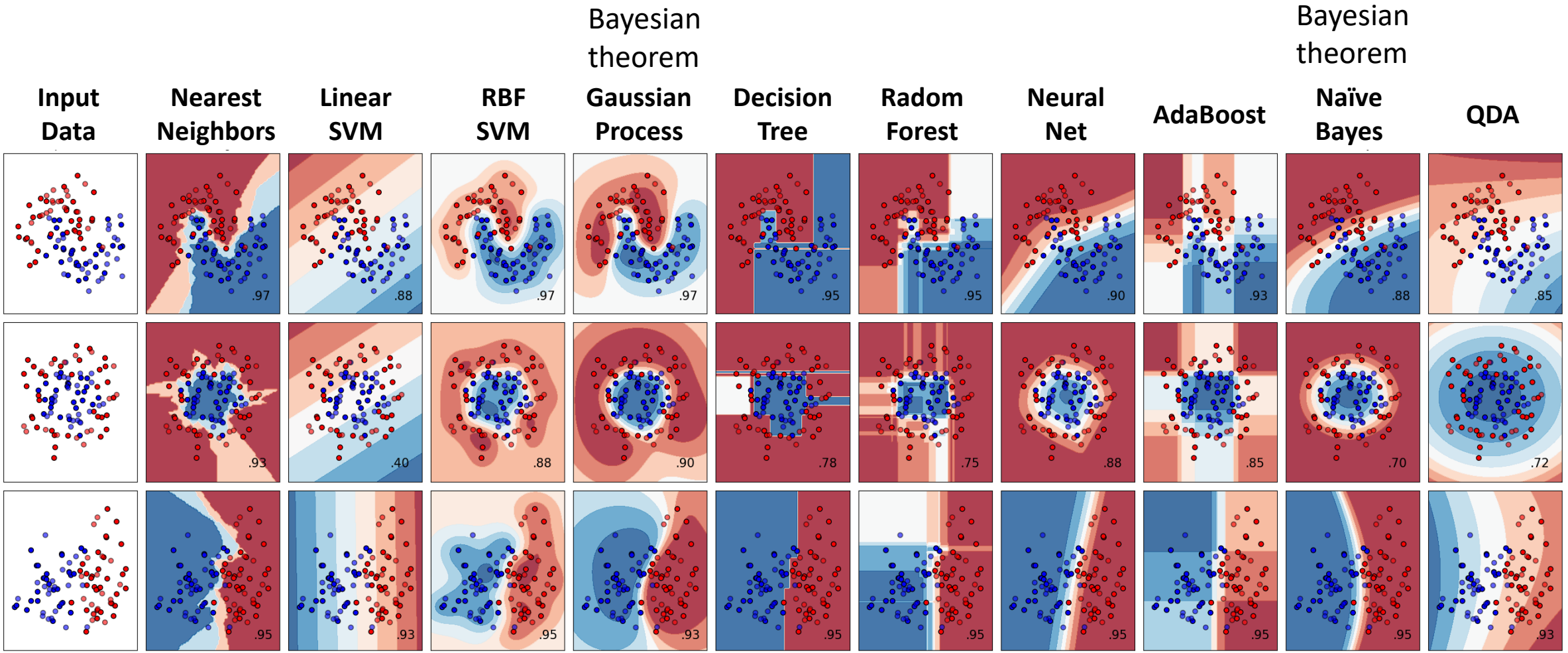
- Supervised Learning: Uses labeled data to train models (e.g., regression, classification)
- Unsupervised Learning: Uses unlabeled data to find patterns (e.g., clustering, dimensionality reduction)



Classification



Classification

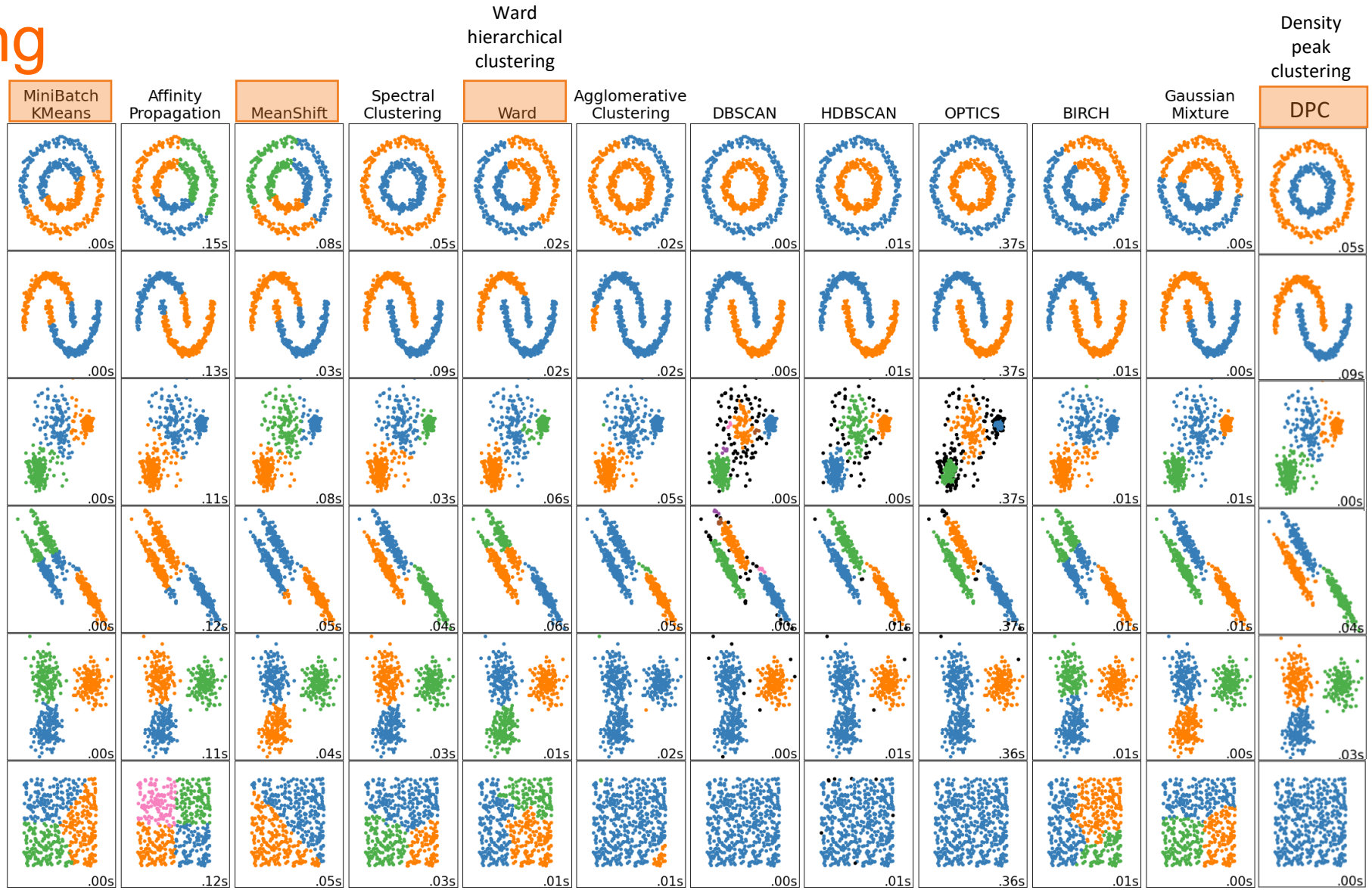


Source: https://scikit-learn.org/stable/

Clustering



Clustering



Source: https://scikit-learn.org/stable/modules/clustering

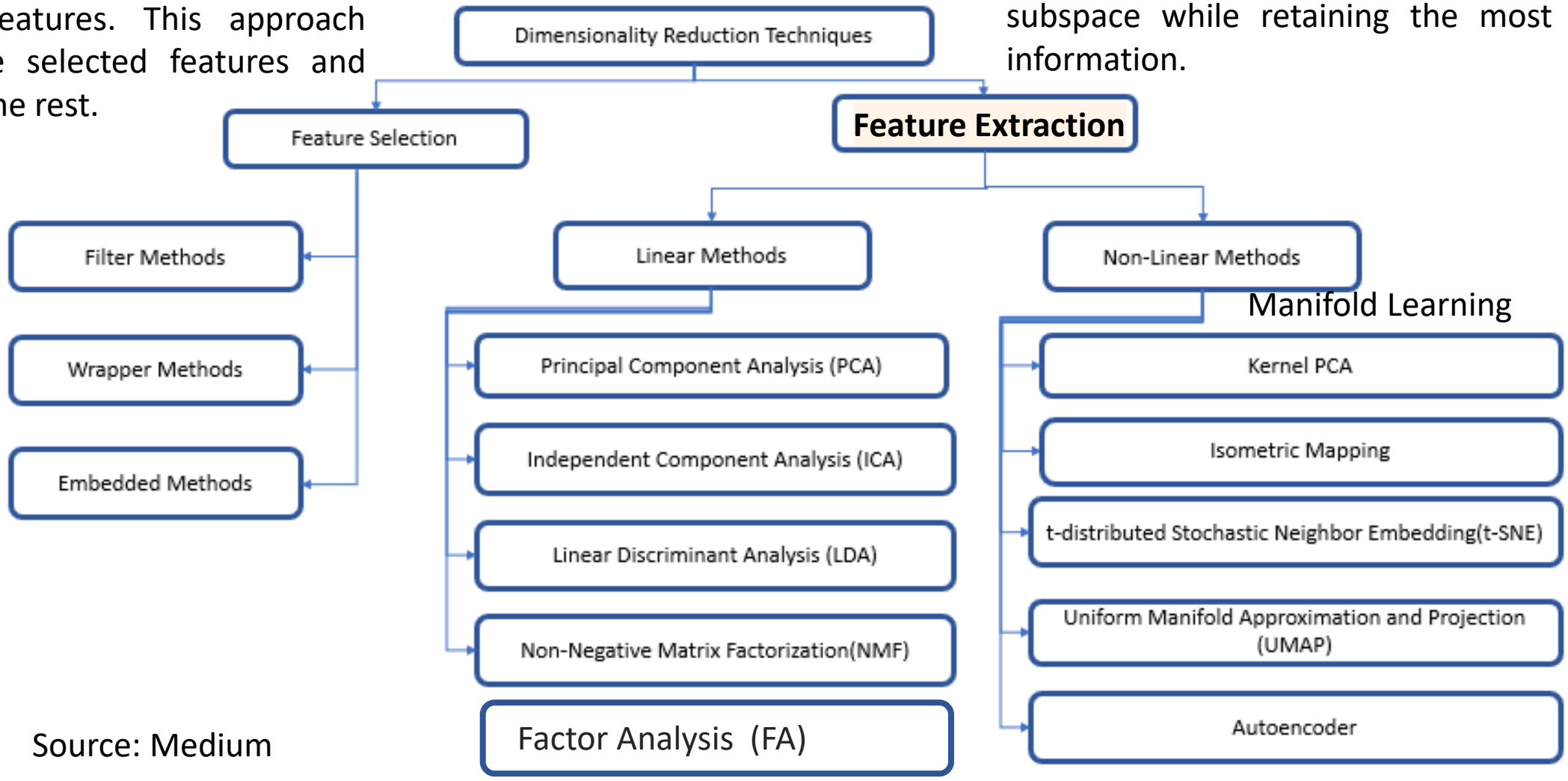
Dimensionality Reduction



Dimensionality Reduction

Involves selecting a subset of the original features. This approach keeps the selected features and discards the rest.

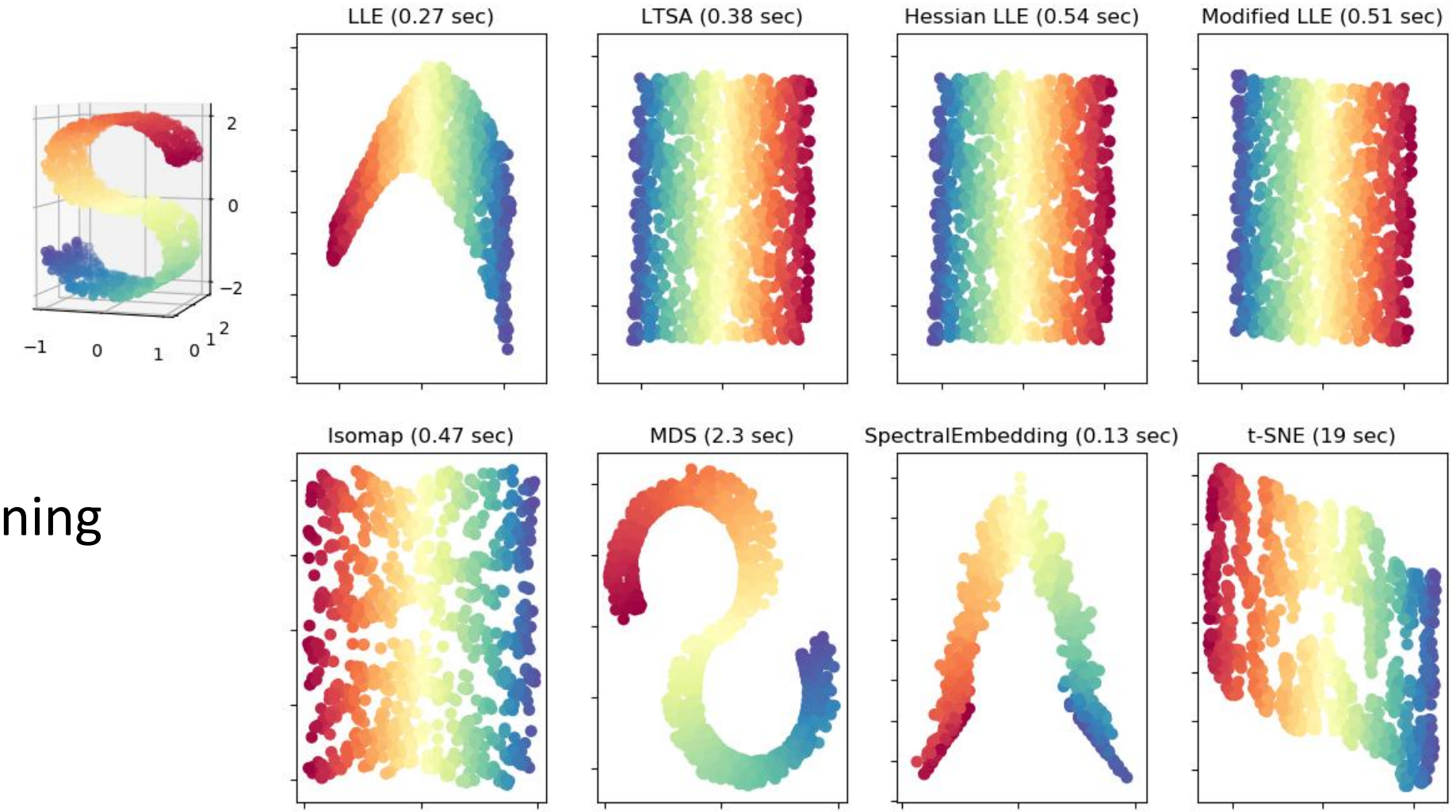
Involves transforming the original features into a lower-dimensional space. This transformation is typically done by projecting the data onto a new subspace while retaining the most important information.



Source: Medium

Dimensionality Reduction

Source: https://scikit-learn.org/



Manifold Learning

Regression



Regression



Source: https://gist.github.com/kwinkunks/48f86f4c368f5003d13dee734a16800c

scikit-learn: Machine Learning in Python

Install User Guide API Examples Community More 1.5.1 (stable)

scikit-learn

Machine Learning in Python

Getting Started | Release Highlights for 1.5

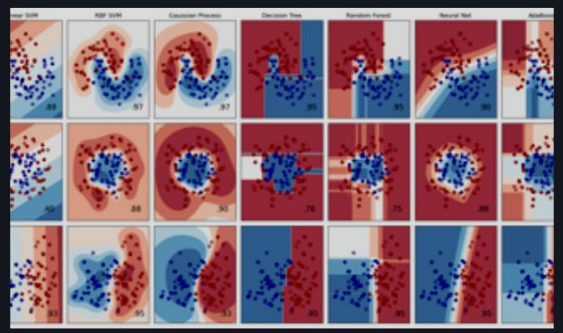
- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Source: https://scikit-learn.org/stable/index.html

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.
Algorithms: Gradient boosting, nearest neighbors, random forest, logistic regression, and more...

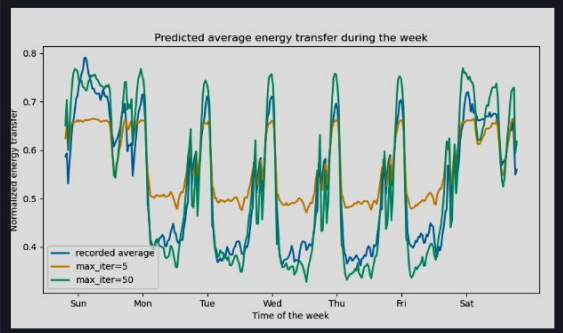


Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, stock prices.
Algorithms: Gradient boosting, nearest neighbors, random forest, ridge, and more...

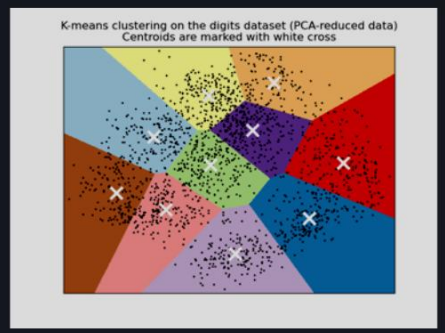


Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, grouping experiment outcomes.
Algorithms: k-Means, HDBSCAN, hierarchical clustering, and more...



Examples

scikit-learn: Machine Learning in Python

Install User Guide API Examples Community More 1.5.1 (stable)

scikit-learn

Machine Learning in Python

Getting Started | Release Highlights for 1.5

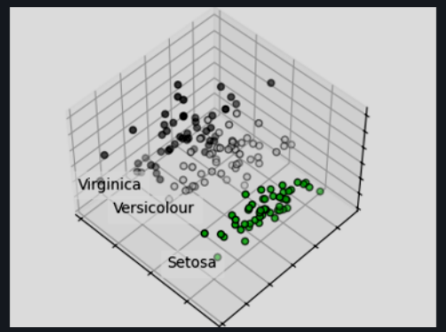
- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Source: https://scikit-learn.org/stable/index.html

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, increased efficiency.
Algorithms: PCA, feature selection, non-negative matrix factorization, and more...

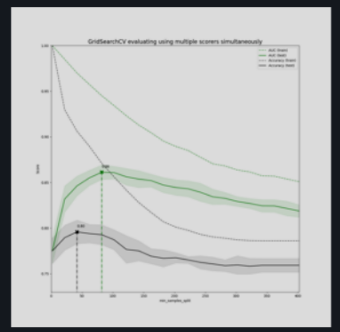


Examples

Model selection

Comparing, validating and choosing parameters and models.

Applications: Improved accuracy via parameter tuning.
Algorithms: Grid search, cross validation, metrics, and more...

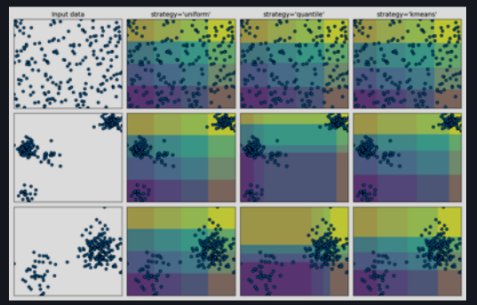


Examples

Preprocessing

Feature extraction and normalization.

Applications: Transforming input data such as text for use with machine learning algorithms.
Algorithms: Preprocessing, feature extraction, and more...



Examples

scikit-learn: Machine Learning in Python

Scikit-learn is a widely used machine learning library in Python that provides a variety of tools for building machine learning models. Here are six main use cases of scikit-learn:

1.Classification:

- Scikit-learn provides various algorithms for classification tasks, where the goal is to predict the category or class of a given data point.
- Algorithms like Support Vector Machines, Random Forest, and K-Nearest Neighbors are commonly used for classification tasks.

2.Regression:

- Scikit-learn offers regression algorithms to predict continuous values.
- Linear regression, Decision Trees, and Gradient Boosting Regressors are examples of regression algorithms available in scikit-learn.

3.Clustering:

- Scikit-learn supports clustering algorithms for unsupervised learning tasks, where the goal is to group similar data points together.
- K-Means, DBSCAN, and Hierarchical clustering are popular clustering algorithms provided by scikit-learn.

4.Dimensionality Reduction:

- Dimensionality reduction techniques are useful for reducing the number of features in a dataset while preserving important information.
- Principal Component Analysis (PCA) and t-Distributed Stochastic Neighbor Embedding (t-SNE) are commonly used techniques available in scikit-learn.



scikit-learn: Machine Learning in Python

Scikit-learn is a widely used machine learning library in Python that provides a variety of tools for building machine learning models. Here are six main use cases of scikit-learn:

5. **Model Selection and Evaluation**:
   - Scikit-learn provides tools for model selection, evaluation, and validation, such as cross-validation, grid search, and metrics for evaluating model performance.
   - These tools help in fine-tuning models and selecting the best performing model for a given task.

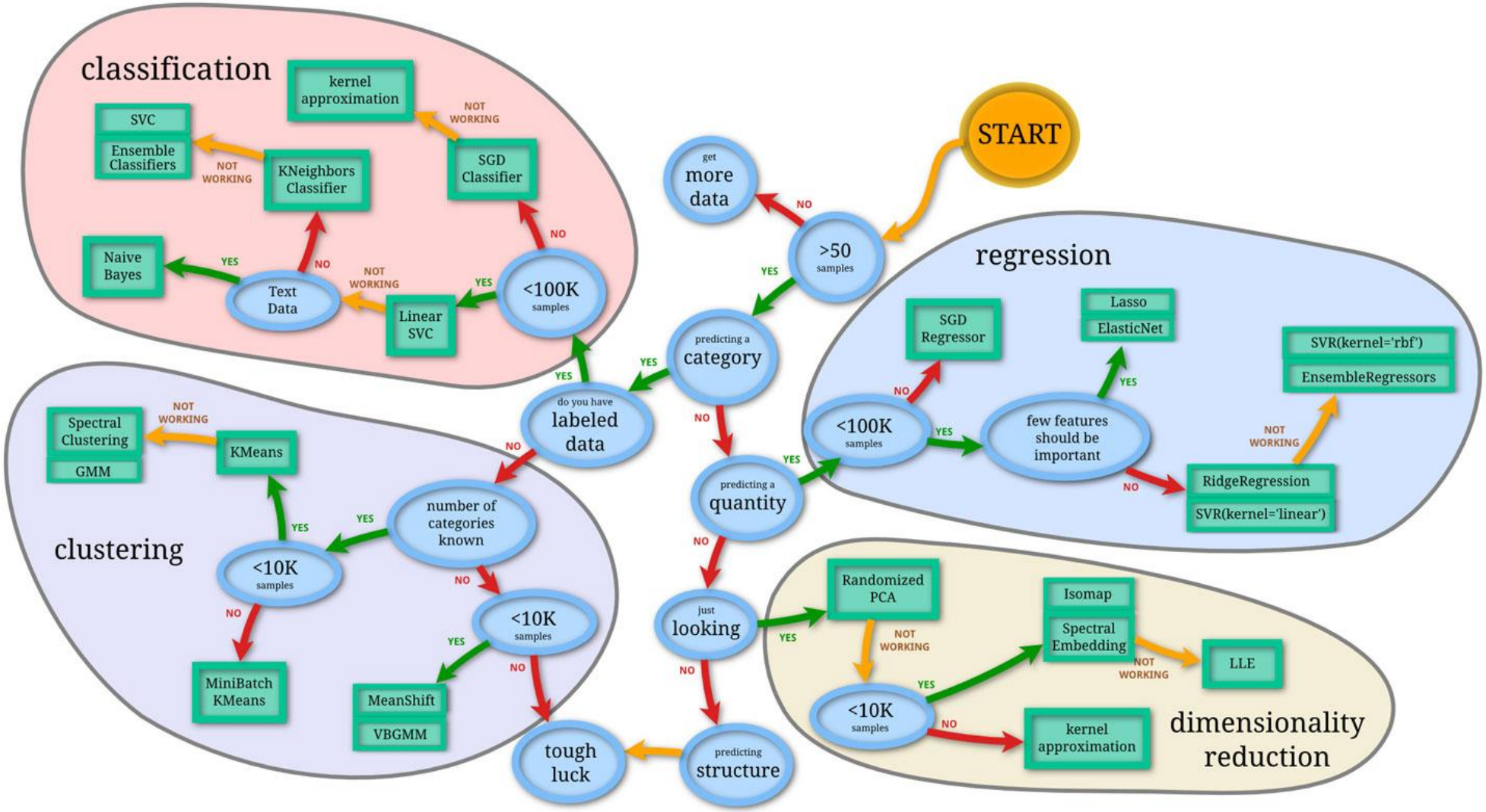
6.Preprocessing and Feature Engineering:

- Scikit-learn offers various preprocessing techniques for feature scaling, normalization, encoding categorical variables, and handling missing values.
- Feature extraction and transformation methods are also available to create new features from existing ones.



scikit-learn: Machine Learning in Python

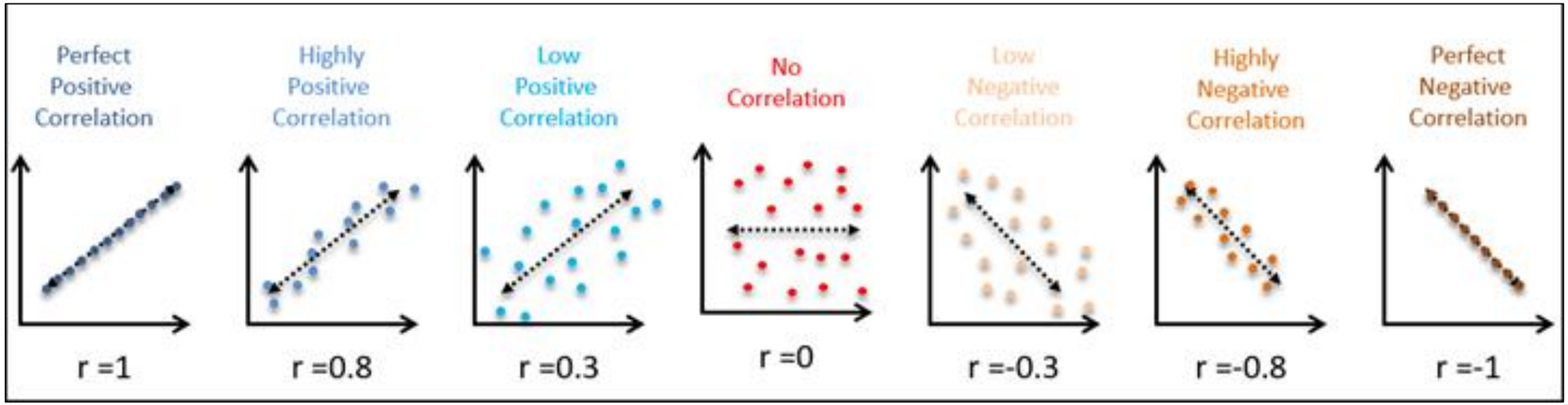
Source: https://scikit-learn.org/stable/



Correlation Analysis

Correlation analysis is a statistical method used to measure and evaluate the strength and direction of the relationship between two or more variables. It helps to determine how closely related the variables are to each other.

Pearson correlation coefficient (*r*) value	Strength	Direction
Greater than .5	Strong	Positive
Between .3 and .5	Moderate	Positive
Between 0 and .3	Weak	Positive
0	None	None
Between 0 and –.3	Weak	Negative
Between –.3 and –.5	Moderate	Negative
Less than –.5	Strong	Negative



Source: https://danawanzer.github.io/stats-with-jamovi/correlation.html

Correlation Analysis

```
import numpy as np
# Sample data: two variables
# For example, let's say we have two variables: X and Y
X = np.array([1, 2, 3, 4, 5])
Y = np.array([2, 4, 6, 8, 10])
# Compute the correlation coefficient
correlation_matrix = np.corrcoef(X, Y)
# Extract the correlation coefficient
correlation_coefficient = correlation_matrix[0, 1]
# Display the results
print("Correlation Matrix:\n", correlation_matrix)
print("Correlation Coefficient between X and Y:", correlation_coefficient)
```

NumPy reference › Routines and objects by topic › Statistics › numpy.corrcoef

On this page

corrcoef

numpy.corrcoef

numpy.corrcoef(*x*, *y=None*, *rowvar=True*, ***, *dtype=None*) [source]

Return Pearson product-moment correlation coefficients.

Please refer to the documentation for **cov** for more detail. The relationship between the correlation coefficient matrix, *R*, and the covariance matrix, *C*, is

$$R_{ij} = \frac{C_{ij}}{\sqrt{C_{ii} C_{jj}}}$$

The values of *R* are between -1 and 1, inclusive.

Parameters:

x : *array_like*

A 1-D or 2-D array containing multiple variables and observations. Each row of *x* represents a variable, and each column a single observation of all those variables. Also see *rowvar* below.

y : *array_like, optional*

An additional set of variables and observations. *y* has the same shape as *x*.

https://numpy.org/devdocs/reference/generated/numpy.corrcoef.html



Correlation Analysis

Matplotlib: subplot example

Faceted Chart



Correlation Analysis

Matplotlib: subplot example

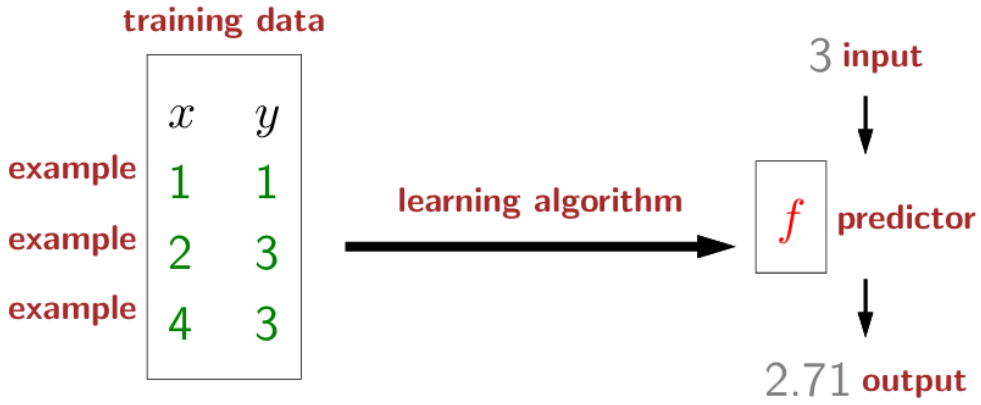
Faceted Chart

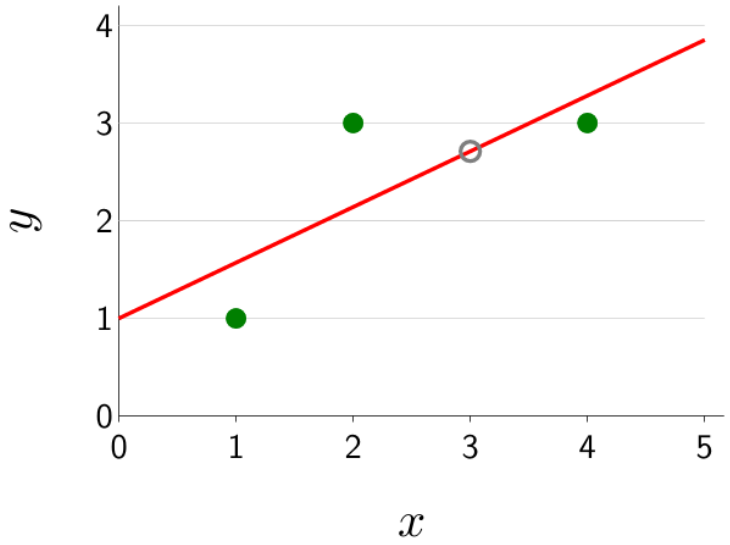


Linear Regression



Simple Linear Regression





Design decisions:

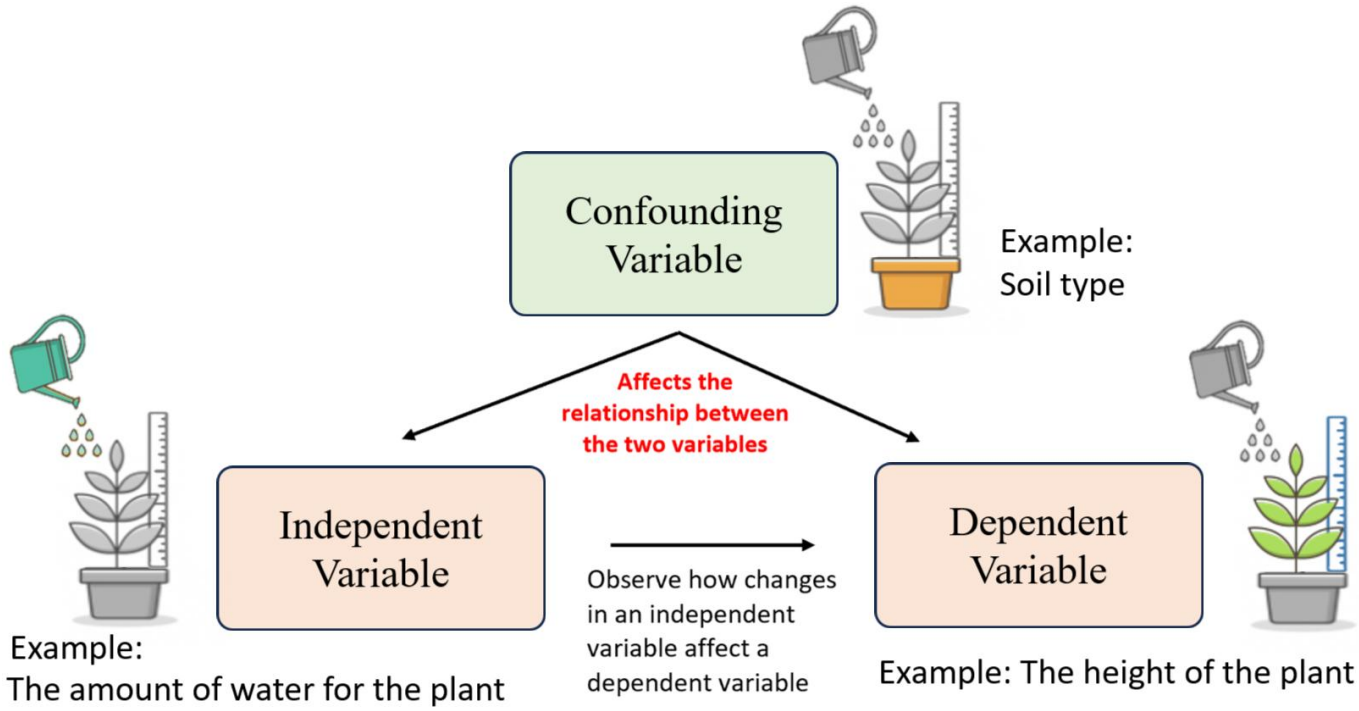
Which predictors are possible? **hypothesis class**

How good is a predictor? **loss function**

How do we compute the best predictor? **optimization algorithm**

Regression

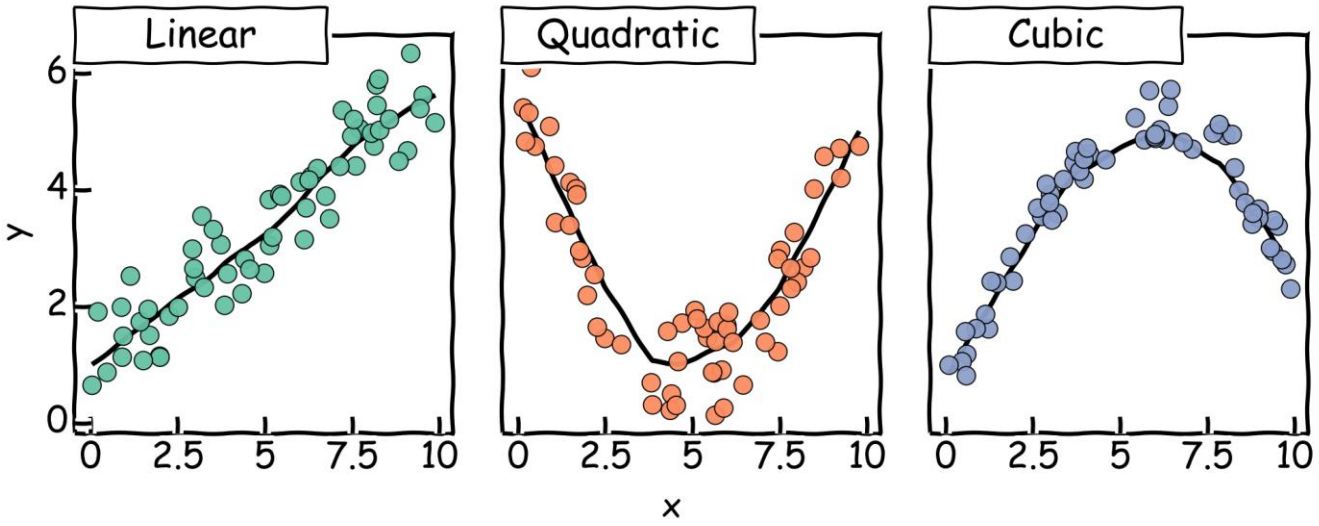
- **Independent Variable**: The independent variable is the variable that is manipulated or changed by the researcher in an experiment. It is called "independent" because its value is not influenced by other variables in the study.
- **Dependent Variable**: The dependent variable is the variable that is measured or observed in an experiment. It is called "dependent" because its value depends on or is influenced by the independent variable.
- **Confounding Variable**: A confounded variable is an extraneous variable that is related to both the independent and dependent variables, making it difficult to determine the true relationship between them.

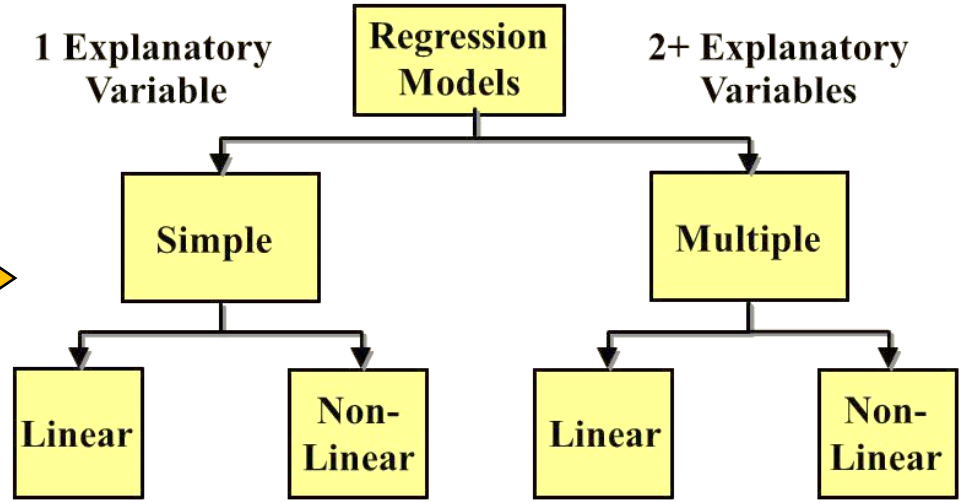


Regression

Regression is a statistical technique used to understand the relationship between one dependent variable and one or more independent variables: $Y=f(X_1, X_2,\ldots, X_n)$

It is commonly employed to predict or estimate the value of the dependent variable based on the values of the independent variables. In regression analysis, the goal is to model the relationship between variables and make predictions or inferences based on that model.





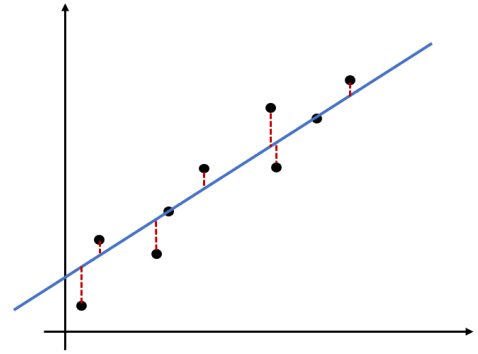
Simple Linear Regression: Ordinary Least Squares (OLS)

Given a dataset $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ consisting of inputs x and outputs y, where $x_i = (x_{i1}; x_{i2}; \dots; x_{id}), y_i \in \mathbb{R}$. The linear regression aims to train a linear model to best fit the output y based on the input x. We aim to learn the relationship to approximate y_i : $y = wx_i + b$. Then, $y \cong y_i$.

The key challenge in linear regression learning lies in determining the parameters w and b to make the fitted output y as close as possible to the true output y_i. In regression tasks, mean squared error is commonly used to measure the loss between predictions and labels. Therefore, the optimization goal in regression tasks is to minimize the mean squared error between the fitted output and the true output.

$$y = wx_i + b$$

$$(w^*, b^*) = \arg\min \sum_{i=1}^{m} (y - y_i)^2 = \arg\min \sum_{i=1}^{m} (wx_i + b - y_i)^2$$



Simple Linear Regression: Ordinary Least Squares (OLS)

To obtain the minimizing parameters $w*$ and $b*$ for w and b, based on the mean square loss function, the first derivatives of w and b can be calculated and equated to zero. The derivation process for w is as follows:

$$\frac{\partial L(w,b)}{\partial w} = \frac{\partial}{\partial w}\left[\sum_{i=1}^{m}(wx_i + b - y_i)^2\right]$$
$$= \sum_{i=1}^{m}\frac{\partial}{\partial w}[(y_i - wx_i - b)^2]$$
$$= \sum_{i=1}^{m}[2\cdot(y_i - wx_i - b)\cdot(-x_i)]$$
$$= \sum_{i=1}^{m}[2\cdot(wx_i^2 - y_i x_i + bx_i)]$$
$$= 2\cdot\left(w\sum_{i=1}^{m}x_i^2 - \sum_{i=1}^{m}y_i x_i + b\sum_{i=1}^{m}x_i\right)$$

$$\frac{\partial L(w,b)}{\partial b} = \frac{\partial}{\partial b}\left[\sum_{i=1}^{m}(wx_i + b - y_i)^2\right]$$
$$= \sum_{i=1}^{m}\frac{\partial}{\partial b}[(y_i - wx_i - b)^2]$$
$$= \sum_{i=1}^{m}[2\cdot(y_i - wx_i - b)\cdot(-1)]$$
$$= \sum_{i=1}^{m}[2\cdot(-y_i + wx_i + b)]$$
$$= 2\cdot(-\sum_{i=1}^{m}y_i + \sum_{i=1}^{m}wx_i + \sum_{i=1}^{m}b)$$
$$= 2(mb - \sum_{i=1}^{m}(y_i - wx_i))$$



Simple Linear Regression: Ordinary Least Squares (OLS)

The best expressions of w and b can be solved as:

$$\frac{\partial L(w,b)}{\partial b} = 2(mb - \sum_{i=1}^{m}(y_i - wx_i))=0 \Rightarrow b^* = \frac{1}{m}\sum_{i=1}^{m}(y_i - wx_i) = \bar{y} - w\bar{x}$$

$$\frac{\partial L(w,b)}{\partial w} = \frac{\partial}{\partial w}\left[\sum_{i=1}^{m}(wx_i + b - y_i)^2\right]$$

$$= \sum_{i=1}^{m}\frac{\partial}{\partial w}[(y_i - wx_i - b)^2]$$

$$= \sum_{i=1}^{m}[2\cdot(y_i - wx_i - b)\cdot(-x_i)]$$

$$= \sum_{i=1}^{m}[2\cdot(y_i - wx_i - \bar{y} + w\bar{x})\cdot(-x_i)]$$

$$= -2\sum_{i=1}^{m}[(y_i - \bar{y} + w(\bar{x} - x_i))\cdot x_i]=0$$

$$\Rightarrow \sum_{i=1}^{m}(y_i - \bar{y} + w(\bar{x} - x_i)) = 0$$

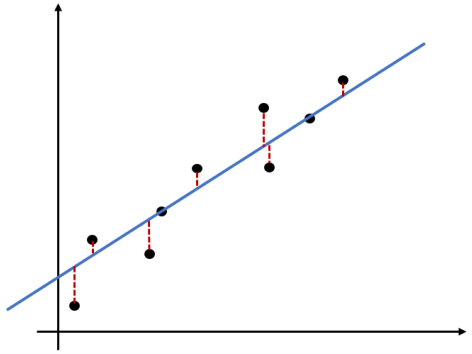
$$\sum_{i=1}^{m}(y_i - \bar{y}) = -w\sum_{i=1}^{m}(\bar{x} - x_i)$$

$$w^* = \frac{\sum_{i=1}^{m}(y_i - \bar{y})}{\sum_{i=1}^{m}(x_i - \bar{x})} = \frac{\sum_{i=1}^{m}(y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^{m}(x_i - \bar{x})^2}$$



Simple Linear Regression: Ordinary Least Squares (OLS)

The best expressions of w and b can be solved as:



$$\frac{\partial L(w,b)}{\partial b} = 2(mb - \sum_{i=1}^{m}(y_i - wx_i))=0 \Rightarrow b^* = \frac{1}{m}\sum_{i=1}^{m}(y_i - wx_i) = \bar{y} - w\bar{x}$$

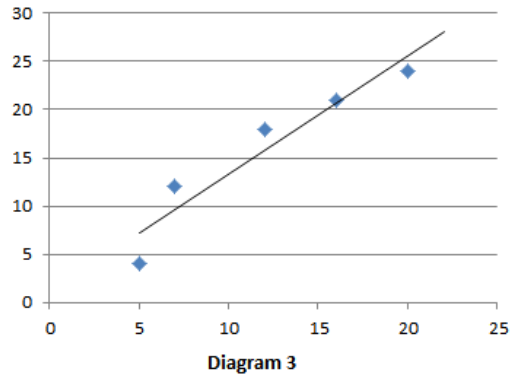
$$\frac{\partial L(w,b)}{\partial w} = 2 \cdot \left(w\sum_{i=1}^{m} x_i^2 - \sum_{i=1}^{m} y_i x_i + b\sum_{i=1}^{m} x_i\right)=0 \Rightarrow w^* = \frac{\sum_{i=1}^{m} y_i(x_i - \bar{x})}{\sum_{i=1}^{m} x_i^2 - \frac{1}{m}\left(\sum_{i=1}^{m} x_i\right)^2}$$

$$= \frac{\sum_{i=1}^{m}(y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^{m}(x_i - \bar{x})^2}$$

Simple Linear Regression: Ordinary Least Squares (OLS)

The effect of water x on the plant growth rate y was measured:

water, x	5	7	12	16	20
growth rate y:	40	120	180	210	240



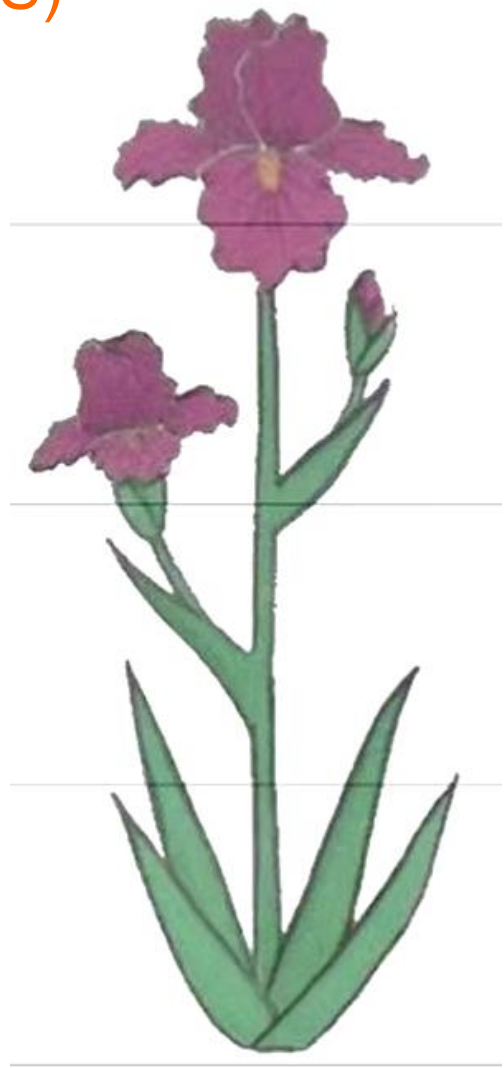
Find the equation of the regression line.

$$y = wx_i + b$$

$$b^* = \frac{1}{m}\sum_{i=1}^{m}(y_i - wx_i) = \bar{y} - w\bar{x}$$

$$w^* = \frac{\sum_{i=1}^{m} y_i(x_i - \bar{x})}{\sum_{i=1}^{m} x_i^2 - \frac{1}{m}\left(\sum_{i=1}^{m} x_i\right)^2}$$

$$= \frac{\sum_{i=1}^{m}(y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^{m}(x_i - \bar{x})^2}$$



Simple Linear Regression: Ordinary Least Squares (OLS)

The effect of water x on the plant growth rate y was measured:

water, x	5	7	12	16	20
growth rate y:	40	120	180	210	240

Find the equation of the regression line.

$$y = wx_i + b$$

$$b^* = \frac{1}{m}\sum_{i=1}^{m}(y_i - wx_i) = \bar{y} - w\bar{x}$$

$$w^* = \frac{\sum_{i=1}^{m} y_i(x_i - \bar{x})}{\sum_{i=1}^{m} x_i^2 - \frac{1}{m}\left(\sum_{i=1}^{m} x_i\right)^2} = \frac{\sum_{i=1}^{m}(y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^{m}(x_i - \bar{x})^2}$$

$$\bar{x} = \frac{\sum x}{n} = \frac{5+7+12+16+20}{5} = \frac{60}{5} = 12,$$

$$\bar{y} = \frac{\sum y}{n} = \frac{40+120+180+210+240}{5} = \frac{790}{5} = 158.$$

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
5	40	$5 - 12 = -7$	$40 - 158 = -118$	$-7 \times -118 = 826$	$-7^2 = 49$
7	120	$7 - 12 = -5$	$120 - 158 = -38$	$-5 \times -38 = 190$	$-5^2 = 25$
12	180	$12 - 12 = 0$	$180 - 158 = 22$	$0 \times 22 = 0$	$0^2 = 0$
16	210	$16 - 12 = 4$	$210 - 158 = 52$	$4 \times 52 = 208$	$4^2 = 16$
20	240	$20 - 12 = 8$	$240 - 158 = 82$	$8 \times 82 = 656$	$8^2 = 64$
$\sum x = 60$	$\sum y = 790$			$\sum(x_i - \bar{x})(y_i - \bar{y}) = 1880$	$\sum(x_i - \bar{x})^2 = 154$

$$b = \frac{S_{xy}}{S_{xx}} = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2} = \frac{1880}{154} = 12.20779\ldots = 12.208 \text{ (3.d.p.)}$$

$$w = \bar{y} - b\bar{x} = 158 - 12.208 \times 12 = 11.506\ldots = 11.506 \text{ (3.d.p.).}$$



Regression Evaluation Metrics

- **Error-based metrics**, including Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Mean Absolute Percentage Error (MAPE), quantify the differences between predicted and actual values.

- **Mean Squared Error (MSE) and Root Mean Squared Error (RMSE)**: MSE and RMSE quantify the average squared difference between the predicted values and the actual values. Lower values of MSE and RMSE indicate better model performance, as they suggest that the model's predictions are closer to the true values. RMSE is particularly useful as it has the same unit of measurement as the dependent variable, making it easier to interpret.

$$MSE = \frac{1}{N}\Sigma_{i=1}^{N}(y_i - \hat{y}_i)^2 \qquad RMSE = \sqrt{\frac{1}{N}\Sigma_{i=1}^{N}(y_i - \hat{y}_i)^2}$$

- **Mean Absolute Error (MAE)**: MAE calculates the average absolute difference between the predicted values and the actual values. Like MSE and RMSE, lower values of MAE indicate better model performance. MAE is less sensitive to outliers compared to MSE, making it a suitable metric when outliers are present in the data.

$$MAE = \frac{1}{N}\Sigma_{i=1}^{N}|y_i - \hat{y}_i|$$



Regression Evaluation Metrics

- **Coefficient of Determination metrics**, such as R-squared (R^2), assess the proportion of variance explained by the model. R-squared measures the proportion of variance in the dependent variable that is explained by the regression model. It ranges from 0 to 1, with higher values indicating a better fit. R-squared can help assess how well the model captures the variability in the data and how much of the response variable's variation is accounted for by the predictors.

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\Sigma_{i=1}^{N}(y_i - \hat{y}_i)^2}{\Sigma_{i=1}^{N}(y_i - \bar{y}_i)^2}$$

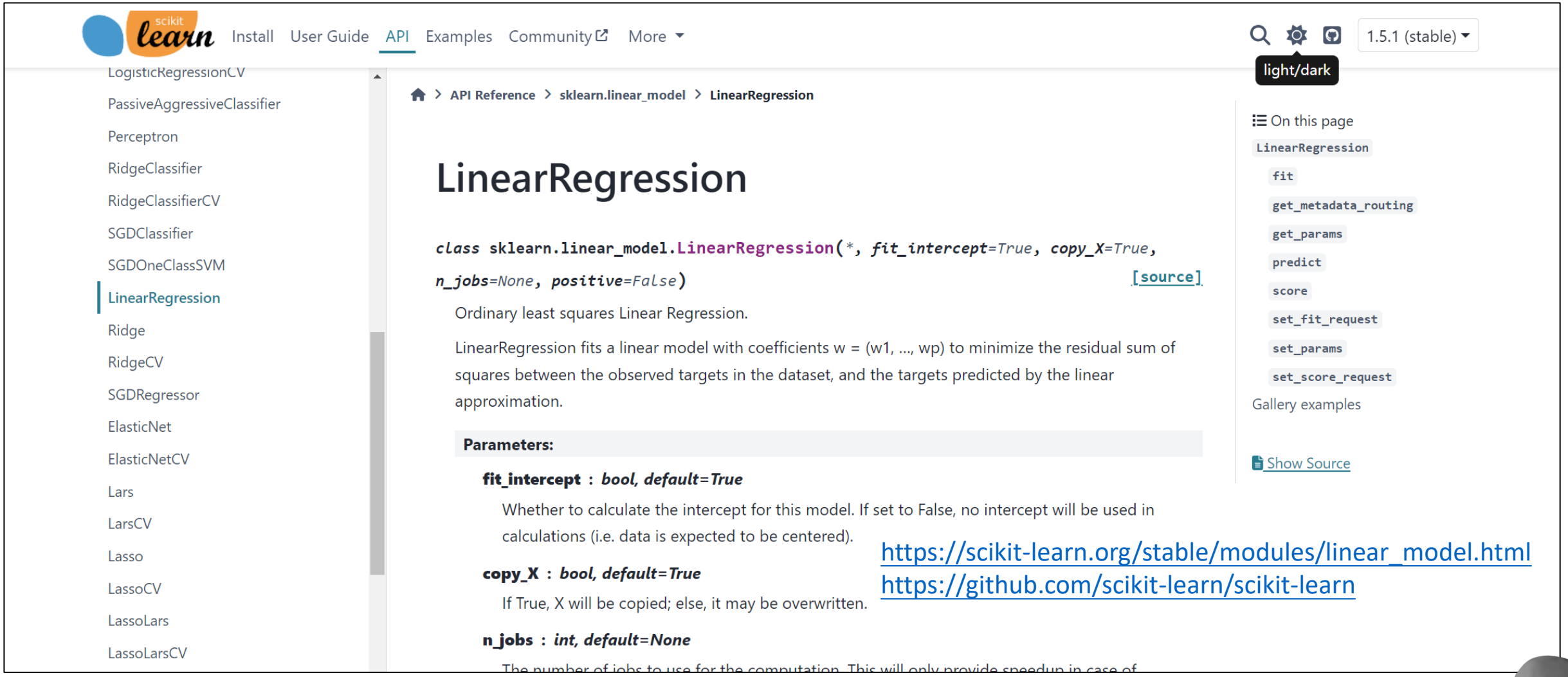
- **Adjusted R-squared** is a modified form of R-squared that considers the number of predictors in a model. Itis a valuable metric for assessing the goodness of fit of regression models, especially when comparing models with different numbers of predictors. Generally positive, the adjusted R-squared is designed to be lower than the R-squared value, reflecting a balance between model complexity and explanatory power.

$$Adjusted\ R^2 = 1 - \frac{(1 - R^2)(N - 1)}{N - p - 1}$$

R^2 Sample R-Squared
N Total Sample Size
p Number of independent variable



scikit-learn: Machine Learning in Python



https://scikit-learn.org/stable/modules/linear_model.html
https://github.com/scikit-learn/scikit-learn

scikit-learn: Linear Regression

LinearRegression

```
class sklearn.linear_model.LinearRegression(*, fit_intercept=True, copy_X=True,
n_jobs=None, positive=False)                                              [source]
```

Ordinary least squares Linear Regression.

LinearRegression fits a linear model with coefficients w = (w1, ..., wp) to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation.

Parameters:

fit_intercept : *bool, default=True*

Whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e. data is expected to be centered).

copy_X : *bool, default=True*

If True, X will be copied; else, it may be overwritten.

n_jobs : *int, default=None*

The number of jobs to use for the computation. This will only provide speedup in case of sufficiently large problems, that is if firstly `n_targets > 1` and secondly `X` is sparse or if `positive` is set to `True`. `None` means 1 unless in a `joblib.parallel_backend` context. `-1` means using all processors. See Glossary for more details.

positive : *bool, default=False*

When set to `True`, forces the coefficients to be positive. This option is only supported for dense arrays.

Attributes:

coef_ : *array of shape (n_features,) or (n_targets, n_features)*

Estimated coefficients for the linear regression problem. If multiple targets are passed during the fit (y 2D), this is a 2D array of shape (n_targets, n_features), while if only one target is passed, this is a 1D array of length n_features.

rank_ : *int*

Rank of matrix `X`. Only available when `X` is dense.

singular_ : *array of shape (min(X, y),)*

Singular values of `X`. Only available when `X` is dense.

intercept_ : *float or array of shape (n_targets,)*

Independent term in the linear model. Set to 0.0 if `fit_intercept = False`.

n_features_in_ : *int*

Number of features seen during fit.

Added in version 0.24.

feature_names_in_ : *ndarray of shape (`n_features_in_` ,)*

Names of features seen during fit. Defined only when `X` has feature names that are all strings.

Added in version 1.0.



scikit-learn: Linear Regression Evaluation

Regression metrics

User guide. See the Regression metrics section for further details.

`d2_absolute_error_score`	D^2 regression score function, fraction of absolute error explained.
`d2_pinball_score`	D^2 regression score function, fraction of pinball loss explained.
`d2_tweedie_score`	D^2 regression score function, fraction of Tweedie deviance explained.
`explained_variance_score`	Explained variance regression score function.
`max_error`	The max_error metric calculates the maximum residual error.
`mean_absolute_error`	Mean absolute error regression loss.
`mean_absolute_percentage_error`	Mean absolute percentage error (MAPE) regression loss.
`mean_gamma_deviance`	Mean Gamma deviance regression loss.
`mean_pinball_loss`	Pinball loss for quantile regression.
`mean_poisson_deviance`	Mean Poisson deviance regression loss.
`mean_squared_error`	Mean squared error regression loss.
`mean_squared_log_error`	Mean squared logarithmic error regression loss.
`mean_tweedie_deviance`	Mean Tweedie deviance regression loss.
`median_absolute_error`	Median absolute error regression loss.
`r2_score`	R^2 (coefficient of determination) regression score function.
`root_mean_squared_error`	Root mean squared error regression loss.
`root_mean_squared_log_error`	Root mean squared logarithmic error regression loss.



scikit-learn: Linear Regression Evaluation

`sklearn.metrics.r2_score(y_true, y_pred, *, sample_weight=None, multioutput='uniform_average', force_finite=True)` [source]

R^2 (coefficient of determination) regression score function.

Best possible score is 1.0 and it can be negative (because the model can be arbitrarily worse). In the general case when the true y is non-constant, a constant model that always predicts the average y disregarding the input features would get a R^2 score of 0.0.

In the particular case when `y_true` is constant, the R^2 score is not finite: it is either `NaN` (perfect predictions) or `-Inf` (imperfect predictions). To prevent such non-finite numbers to pollute higher-level experiments such as a grid search cross-validation, by default these cases are replaced with 1.0 (perfect predictions) or 0.0 (imperfect predictions) respectively. You can set `force_finite` to `False` to prevent this fix from happening.

Note: when the prediction residuals have zero mean, the R^2 score is identical to the `Explained Variance score`.

Read more in the User Guide.

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.r2_score.html#sklearn.metrics.r2_score

Parameters:

y_true : *array-like of shape (n_samples,) or (n_samples, n_outputs)*

Ground truth (correct) target values.

y_pred : *array-like of shape (n_samples,) or (n_samples, n_outputs)*

Estimated target values.

sample_weight : *array-like of shape (n_samples,), default=None*

Sample weights.

multioutput : *{'raw_values', 'uniform_average', 'variance_weighted'}, array-like of shape (n_outputs,) or None, default='uniform_average'*

Defines aggregating of multiple output scores. Array-like value defines weights used to average scores. Default is "uniform_average".

'raw_values' :

Returns a full set of scores in case of multioutput input.

'uniform_average' :

Scores of all outputs are averaged with uniform weight.

'variance_weighted' :

Scores of all outputs are averaged, weighted by the variances of each individual output.

`sklearn.metrics.root_mean_squared_error(y_true, y_pred, *, sample_weight=None, multioutput='uniform_average')` [source]

Root mean squared error regression loss.

Read more in the User Guide.

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.root_mean_squared_error.html#sklearn.metrics.root_mean_squared_error

Added in version 1.4.

Parameters:

y_true : *array-like of shape (n_samples,) or (n_samples, n_outputs)*

Ground truth (correct) target values.

y_pred : *array-like of shape (n_samples,) or (n_samples, n_outputs)*

Estimated target values.

sample_weight : *array-like of shape (n_samples,), default=None*

Sample weights.

multioutput : *{'raw_values', 'uniform_average'} or array-like of shape (n_outputs,), default='uniform_average'*

Defines aggregating of multiple output values. Array-like value defines weights used to average errors.

'raw_values' :

Returns a full set of errors in case of multioutput input.

'uniform_average' :

Errors of all outputs are averaged with uniform weight.

Returns:

loss : *float or ndarray of floats*

A non-negative floating point value (the best value is 0.0), or an array of floating point values, one for each individual target.



Python-Simple Linear Regression

```python
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

# Create a DataFrame with 10 data samples for X and Y variables
df = pd.DataFrame({
    'x': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'y': [2, 4, 6, 7, 9, 11, 13, 15, 17, 19]
})

# Split the dataset into training and testing sets
X = df[['x']]
y = df['y']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and fit the model
model = LinearRegression()
model.fit(X_train, y_train)

# Get predictions
y_pred = model.predict(X_test)

# Calculate MSE
mse = mean_squared_error(y_test, y_pred)

# Calculate RMSE
rmse = mean_squared_error(y_test, y_pred, squared=False)

# Calculate R²
r2 = r2_score(y_test, y_pred)

# Calculate Adjusted R²
n = len(y_test)  # Number of observations in the test set
p = X_test.shape[1]  # Number of predictors
adjusted_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)

# Print the results
print(f"Testing set predictions: {y_pred}")
print(f"MSE: {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²: {r2:.3f}")
print(f"Adjusted R²: {adjusted_r2:.3f}")
```




Multiple Linear Regression

Multiple Linear Regression is a statistical method used to model the relationship between a dependent variable (response) and multiple independent variables (predictors). It extends the concept of simple linear regression by considering the joint effect of multiple predictors on the outcome.

In multiple linear regression, the relationship between the predictors and the response is assumed to be linear, but the model accounts for the influence of multiple predictors simultaneously. The objective is to estimate the regression coefficients that minimize the sum of squared differences between the observed values of the response variable and the predicted values based on the predictors. The multiple linear regression equation can be represented as:

$$y = w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_n * x_n$$

$$(w_0, w_1, \ldots, w_n) = \arg\min \sum_{i=1}^{m} (y - y_i)^2$$



Multiple Linear Regression

$$y = w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_n * x_n$$

$$\boldsymbol{X} = \begin{pmatrix} x_{11} & x_{12} & \ldots & x_{1d} & 1 \\ x_{21} & x_{22} & \ldots & x_{2d} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{md} & 1 \end{pmatrix} = \begin{pmatrix} x_1^{\mathrm{T}} & 1 \\ \vdots & \vdots \\ x_m^{\mathrm{T}} & 1 \end{pmatrix} \quad \boldsymbol{y} = (y_1; y_2; \ldots; y_m)$$

The matrix expression for the optimization objective function of parameters is:

$$\hat{\boldsymbol{w}}^* = \arg\min (\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{w}})^{\mathrm{T}} (\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{w}})$$

Let $L = (\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{w}})^{\mathrm{T}} (\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{w}})$. The derivation process for the parameter $\hat{\boldsymbol{w}}$ is as follows:

$$L = \boldsymbol{y}^{\mathrm{T}}\boldsymbol{y} - \boldsymbol{y}^{\mathrm{T}}\boldsymbol{X}\hat{\boldsymbol{w}} - \hat{\boldsymbol{w}}^{\mathrm{T}}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{y} + \hat{\boldsymbol{w}}^{\mathrm{T}}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{X}\hat{\boldsymbol{w}}$$

Multiple Linear Regression

$$L = \boldsymbol{y}^{\mathrm{T}}\boldsymbol{y} - \boldsymbol{y}^{\mathrm{T}}\boldsymbol{X}\hat{\boldsymbol{w}} - \hat{\boldsymbol{w}}^{\mathrm{T}}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{y} + \hat{\boldsymbol{w}}^{\mathrm{T}}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{X}\hat{\boldsymbol{w}}$$

The derivation process for the parameter $\hat{\boldsymbol{w}}$ is as follows:

$$\frac{\partial L}{\partial \hat{\boldsymbol{w}}} = \frac{\partial \boldsymbol{y}^{\mathrm{T}}\boldsymbol{y}}{\partial \hat{\boldsymbol{w}}} - \frac{\partial \boldsymbol{y}^{\mathrm{T}}\boldsymbol{X}\hat{\boldsymbol{w}}}{\partial \hat{\boldsymbol{w}}} - \frac{\partial \hat{\boldsymbol{w}}^{\mathrm{T}}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{y}}{\partial \hat{\boldsymbol{w}}} + \frac{\partial \hat{\boldsymbol{w}}^{\mathrm{T}}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{X}\hat{\boldsymbol{w}}}{\partial \hat{\boldsymbol{w}}}$$

$$\frac{\partial \boldsymbol{a}^{\mathrm{T}}\boldsymbol{x}}{\partial \boldsymbol{x}} = \frac{\partial \boldsymbol{x}^{\mathrm{T}}\boldsymbol{a}}{\partial \boldsymbol{x}} = \boldsymbol{a}$$

$$+$$

$$\frac{\partial \boldsymbol{x}^{\mathrm{T}}\boldsymbol{A}\boldsymbol{x}}{\partial \boldsymbol{x}} = \left(\boldsymbol{A} + \boldsymbol{A}^{\mathrm{T}}\right)\boldsymbol{x}$$

By using the **matrix differentiation rule**, we have:

$$\frac{\partial L}{\partial \hat{\boldsymbol{w}}} = 0 - \boldsymbol{X}^{\mathrm{T}}\boldsymbol{y} - \boldsymbol{X}^{\mathrm{T}}\boldsymbol{y} + \left(\boldsymbol{X}^{\mathrm{T}}\boldsymbol{X} + \boldsymbol{X}^{\mathrm{T}}\boldsymbol{X}\right)\hat{\boldsymbol{w}} = 2\boldsymbol{X}^{\mathrm{T}}\left(\boldsymbol{X}\hat{\boldsymbol{w}} - \boldsymbol{y}\right)$$

When the matrix $\boldsymbol{X}^{\mathrm{T}}\boldsymbol{X}$ is full rank or positive definite, setting the above equation to 0, the parameters can be solved as:

$$\hat{\boldsymbol{w}}^{*} = \left(\boldsymbol{X}^{\mathrm{T}}\boldsymbol{X}\right)^{-1}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{y}$$



Python-Multiple Linear Regression

```python
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

# Create a DataFrame with 10 data samples for X and Y variables
df = pd.DataFrame({
    'x1': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'x2': [2, 4, 6, 8, 10, 12, 14, 16, 18, 20],
    'x3': [3, 4, 2, 5, 4, 6, 5, 7, 6, 8],
    'y': [3, 5, 7, 9, 11, 13, 15, 17, 19, 21]
})

# Split the dataset into training and testing sets
X = df[['x1', 'x2', 'x3']]
y = df['y']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and fit the model
model = LinearRegression()
model.fit(X_train, y_train)

# Get predictions
y_pred = model.predict(X_test)

# Calculate MSE
mse = mean_squared_error(y_test, y_pred)

# Calculate RMSE
rmse = mean_squared_error(y_test, y_pred, squared=False)

# Calculate R²
r2 = r2_score(y_test, y_pred)

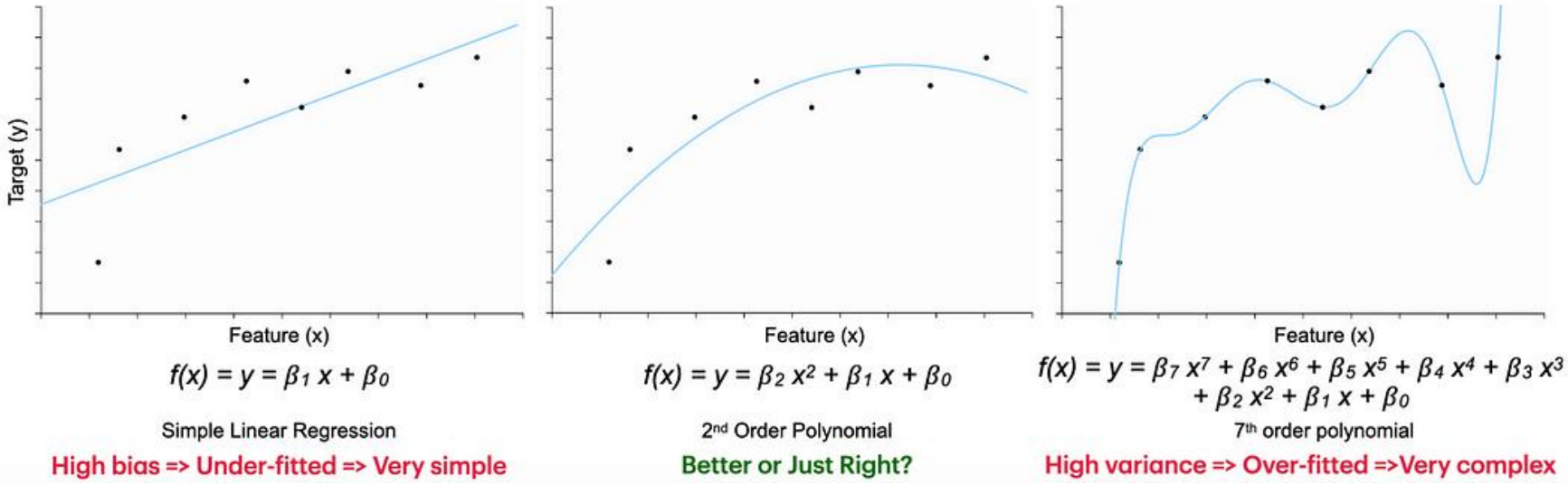
# Calculate Adjusted R²
n = len(y_test)  # Number of observations in the test set
p = X_test.shape[1]  # Number of predictors
adjusted_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)

# Print the results
print(f"Testing set predictions: {y_pred}")
print(f"MSE: {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²: {r2:.3f}")
print(f"Adjusted R²: {adjusted_r2:.3f}")
```




Linear Regression Techniques

- **Underfitting:** Underfitting occurs when the model doesn't work well with both training data and testing data (meaning the accuracy of both training & testing datasets is below 50%). A possible solution is applying Data Wrangling (data preprocessing or feature engineering).
- **Good Fit**: A model is a Good Fit when it works well with both training and testing datasets (meaning the accuracy for both datasets is around 70%–85% in general cases). It means that we almost achieved our goal.
- **Overfitting :** Overfitting occurs when the model works very well with the training dataset, but on the testing dataset, it fails (meaning that the accuracy of training data is >90% and testing data is < 65%). Since the model works best only on training data and whenever it faces a new situation during testing, it gives wrong results. It is also called a model with *high variance.* A possible solution is to use the correct regression technique, that is **Ridge** or **Lasso**.



Linear Regression Techniques: LASSO Regression

The optimal parameter estimation expression for the linear regression model is as follows:

$$\hat{\boldsymbol{w}}^* = \left(\boldsymbol{X}^{\mathrm{T}}\boldsymbol{X}\right)^{-1}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{y}$$

Assuming there are *m* training samples and *n* features,

- When ***m>n***, if rank(X)=n, meaning *X* is full rank, then $\boldsymbol{X}^{\mathrm{T}}\boldsymbol{X}$ is invertible, and the linear regression parameter estimation formula can be directly solved. Typically with *m*>*n*, meaning that the number of samples is greater than the number of features.
- If ***m<n***, implying that the number of features is greater than the number of samples, rank(*X*)<*n*, and the matrix *X* is not full rank. In this case, typically with *m*>*n*, meaning that the number of samples is greater than the number of features. is not invertible, and the parameter $\hat{\boldsymbol{w}}^*$ in the parameter estimation formula is not estimable.

The LASSO (Least Absolute Shrinkage and Selection Operator) model provides a solution for such unestimable parameters in linear regression by adding an L1 regularization term to the loss function:

$$\boldsymbol{L}(\boldsymbol{w}) = (\boldsymbol{y} - \boldsymbol{w}\boldsymbol{X})^2 + \boldsymbol{\lambda}||\boldsymbol{w}||_1$$

Where $||\boldsymbol{w}||_1$ 1 represents the 1-norm of the matrix, λ is the coefficient of the 1-norm term.

A norm can be viewed as a function representing a concept of length or distance. For vectors or matrices, common norms include the 0-norm, 1-norm, 2-norm, and *p*-norm. The 0-norm of a matrix counts the number of non-zero elements in the matrix. The 1-norm of a matrix is defined as the sum of the absolute values of all elements in the matrix, while the L2-norm of a matrix refers to the square root of the sum of squares of all elements in the matrix.



Linear Regression Techniques: LASSO Regression

The above expression is equivalent to adding an L1 regularization term to the original linear regression loss function, where λ is also known as the regularization coefficient. From the perspective of preventing overfitting, the regularization term imposes a penalty on the target parameters, preventing the model from becoming overly complex.

During the optimization process, the presence of the regularization term gradually forces the coefficients of unimportant features to zero, thereby retaining essential features and simplifying the model.

$$L(\boldsymbol{w}) = (\boldsymbol{y} - \boldsymbol{wX})^2 + \lambda||\boldsymbol{w}||_1$$

$$\arg_w \min(\boldsymbol{y} - \boldsymbol{wX})^2$$
$$\text{s.t. } \sum |w_{ij}| < s$$

The constraint $\sum |w_{ij}| < s$ implies that the sum of the absolute values of all elements in the weight coefficient matrix is less than a specified constant ss. A smaller value of s leads to more features having their coefficients compressed to zero.



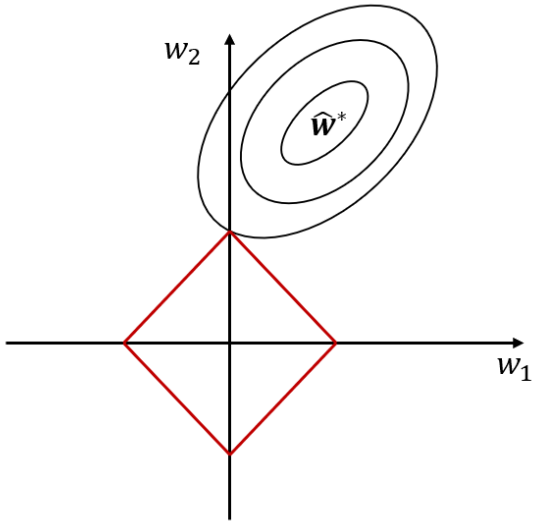
Linear Regression Techniques: LASSO Regression

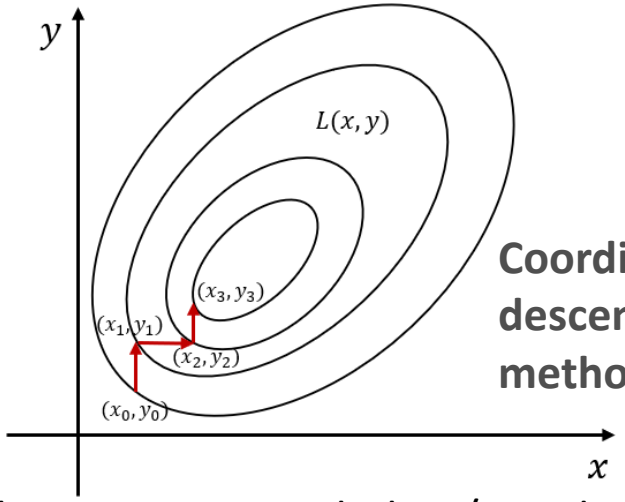
During the optimization process, the presence of the regularization term gradually forces the coefficients of unimportant features to zero, thereby retaining essential features and simplifying the model.

$$\arg_w \min(\boldsymbol{y} - \boldsymbol{wX})^2$$

$$\text{s.t.} \sum |w_{ij}| < s$$

Example: The x-axis and y-axis represent two regression parameters w_1 and w_1 respectively. The red rectangular line represents the L1 regularization constraint of LASSO, $|w_1| + |w_2| \leq s$, while the elliptical region denotes the solution space for the regression parameters. It can be observed that the parameter solution space for LASSO intersects with the y-axis. This intersection implies that the parameter w_1 is being compressed to zero.





Coordinate descent method

https://xavierbourretsicotte.github.io/coordinate_descent.html



Linear Regression Techniques: Ridge Regression

Ridge regression is a model that modifies the linear regression loss function by using the L2-norm as a penalty term. In Ridge regression, $\lambda||\boldsymbol{w}||_2 = \lambda\sum_{i=1}^{n} w_i^2$ is the L2 regularization term. The term $\lambda||\boldsymbol{w}||_2$ is also known as the L2 regularization term.

The principle behind using the L2-norm for regularization is to minimize each element of the parameter matrix, making them approach zero but not exactly zero as with L1 regularization. This approach helps reduce the complexity of the model.

$$L(\boldsymbol{w}) = (\boldsymbol{y} - \boldsymbol{wX})^2 + \lambda||\boldsymbol{w}||_2$$

$$\downarrow$$

$$\arg_w \min(\boldsymbol{y} - \boldsymbol{wX})^2$$

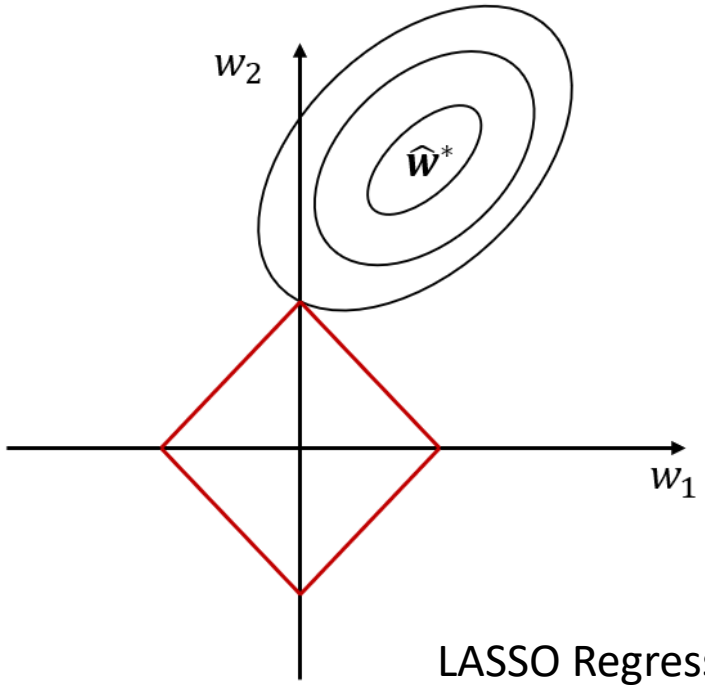
$$\text{s.t.} \sum w_{ij}^2 < s$$

The constraint $\sum w_{ij}^2 < s$ simplies that the sum of the squares of all elements in the weight coefficient matrix is less than a specified constant s.

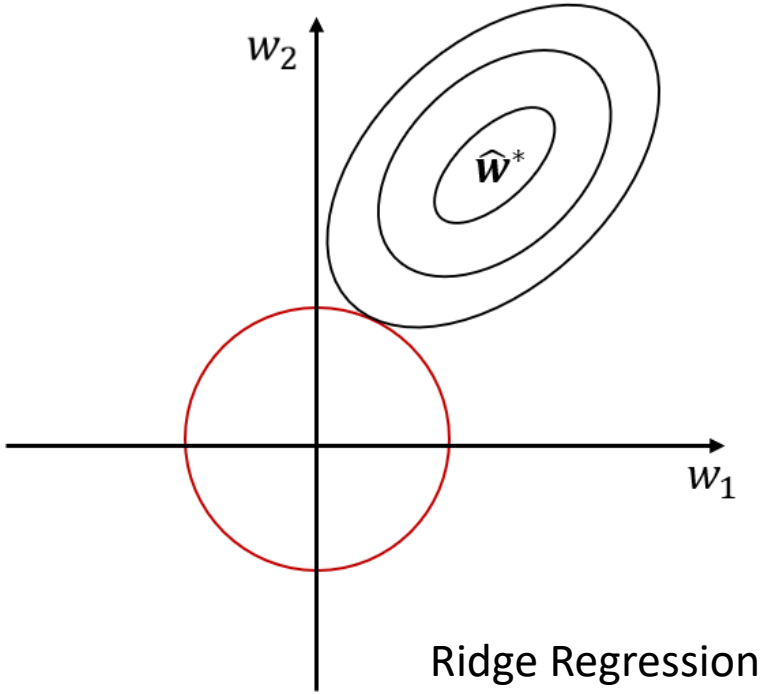
Linear Regression Techniques: Ridge Regression

Ridge Regression: The x-axis and y-axis represent two regression parameters $w1$ and $w2$ respectively. The red circular region represents the L2 regularization constraint of Ridge regression, $w_1^2 + w_2^2 \leq s$, while the elliptical region denotes the solution space for the regression parameters.

It can be observed that the parameter solution space for LASSO intersects with the y-axis, indicating that certain parameters are being compressed to zero, whereas the parameters in Ridge regression are close to zero but not exactly zero.



LASSO Regression



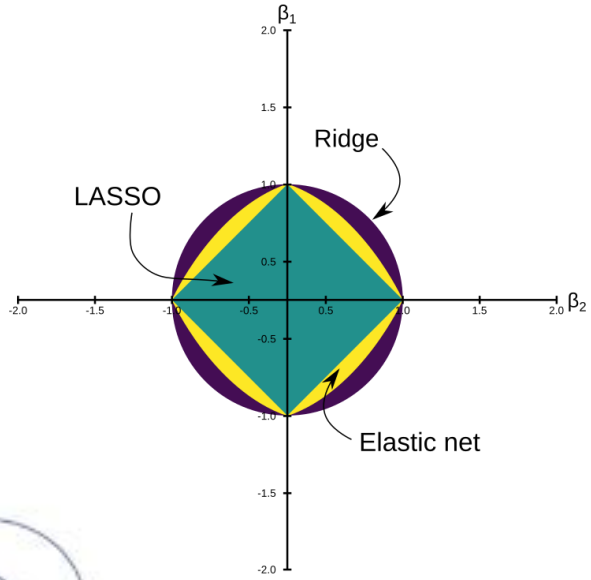
Ridge Regression

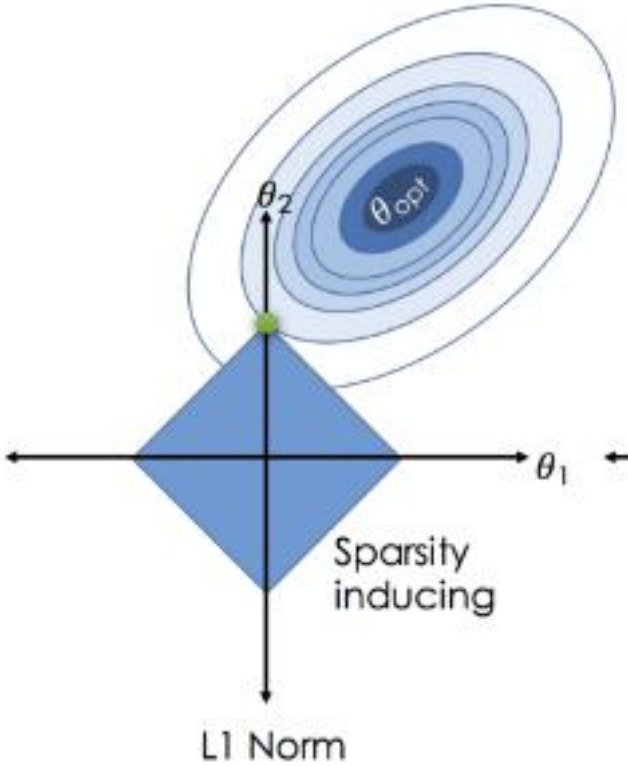
Linear Regression Techniques: Elastic Regression

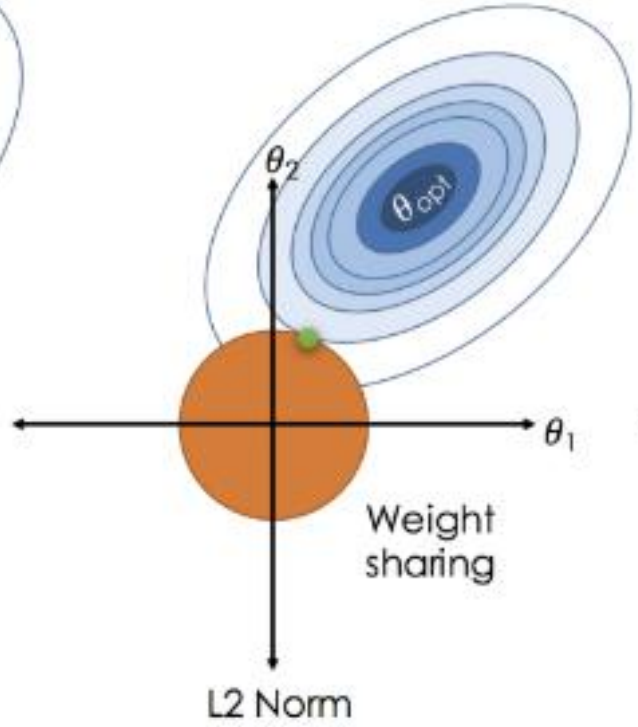
$$J(\theta) = MSE(\theta) + r(2\alpha\Sigma_{i=1}^{n}|\theta_i|) + (1-r)(\frac{\alpha}{m}\Sigma_{i=1}^{n}\theta_i^2)$$

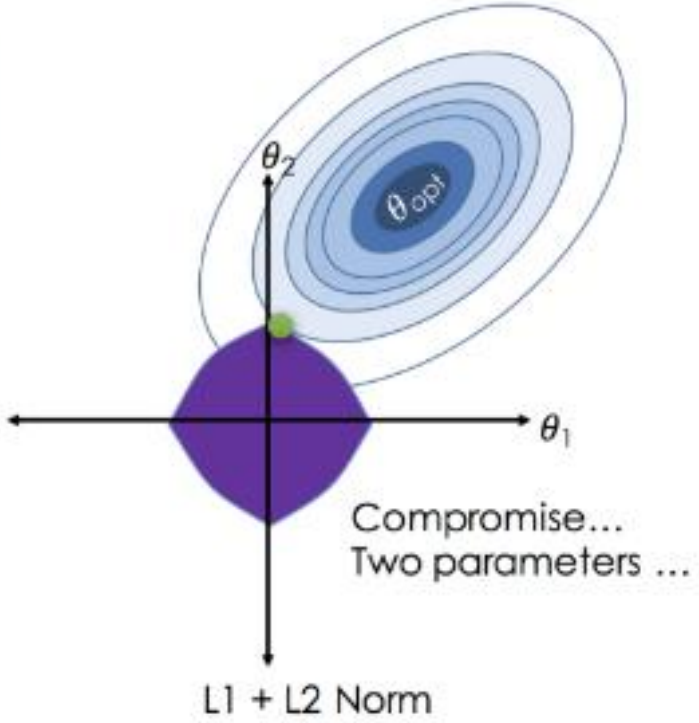
L1 regularization constraint

L2 regularization constraint









Linear Regression Techniques

- From the perspective of interpretability in linear models, both LASSO and Ridge aim to identify key factors influencing the dependent variable while reducing model complexity.
- Mathematically, LASSO and Ridge are viewed as compromise solutions in linear regression models where the parameter estimation formulas are not directly solvable, leading to biased estimations.
- In the context of the three elements of machine learning, LASSO and Ridge can be seen as machine learning methods that utilize linear regression models with regularization terms, minimize the loss function with regularization terms as the strategy, and employ algorithms such as coordinate descent or gradient descent.



Linear Regression Techniques: Stepwise Regression

In statistics, **stepwise regression** is a method of fitting regression models in which the choice of predictive variables is carried out by an automatic procedure. This is **an efficient way to select the most useful explanatory variables**.

$$y = w_0 + w_1 y_1 + w_2 y_2 + \ldots + w_n * y_n$$

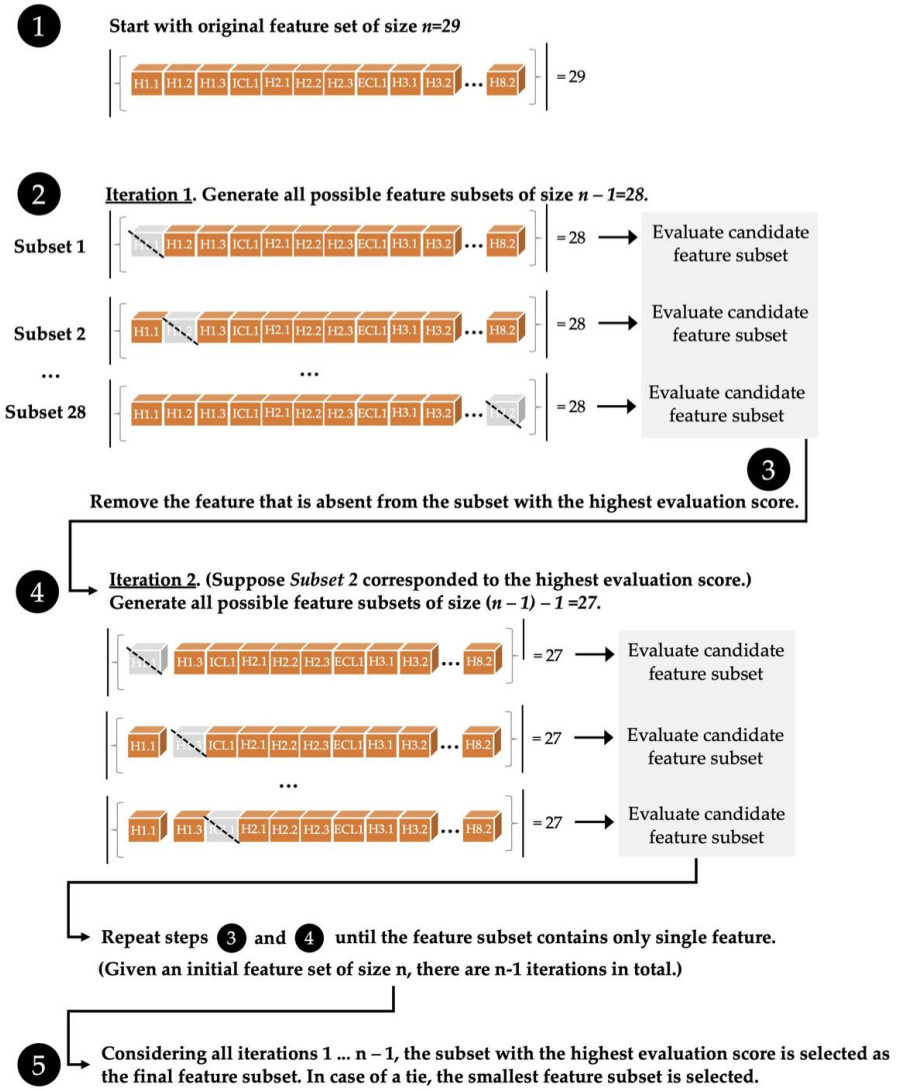
- **Forward selection**, which involves starting with no variables in the model, testing the addition of each variable using a **chosen model fit criterion**, adding the variable (if any) whose inclusion gives the most statistically significant improvement of the fit, and repeating this process until none improves the model to a statistically significant extent.
- **Backward elimination**, which involves starting with all candidate variables, testing the deletion of each variable using a chosen model fit criterion, deleting the variable (if any) whose loss gives the most statistically insignificant deterioration of the model fit, and repeating this process until no further variables can be deleted without a statistically significant loss of fit.
- **Bidirectional elimination**, a combination of the above, testing at each step for variables to be included or excluded.



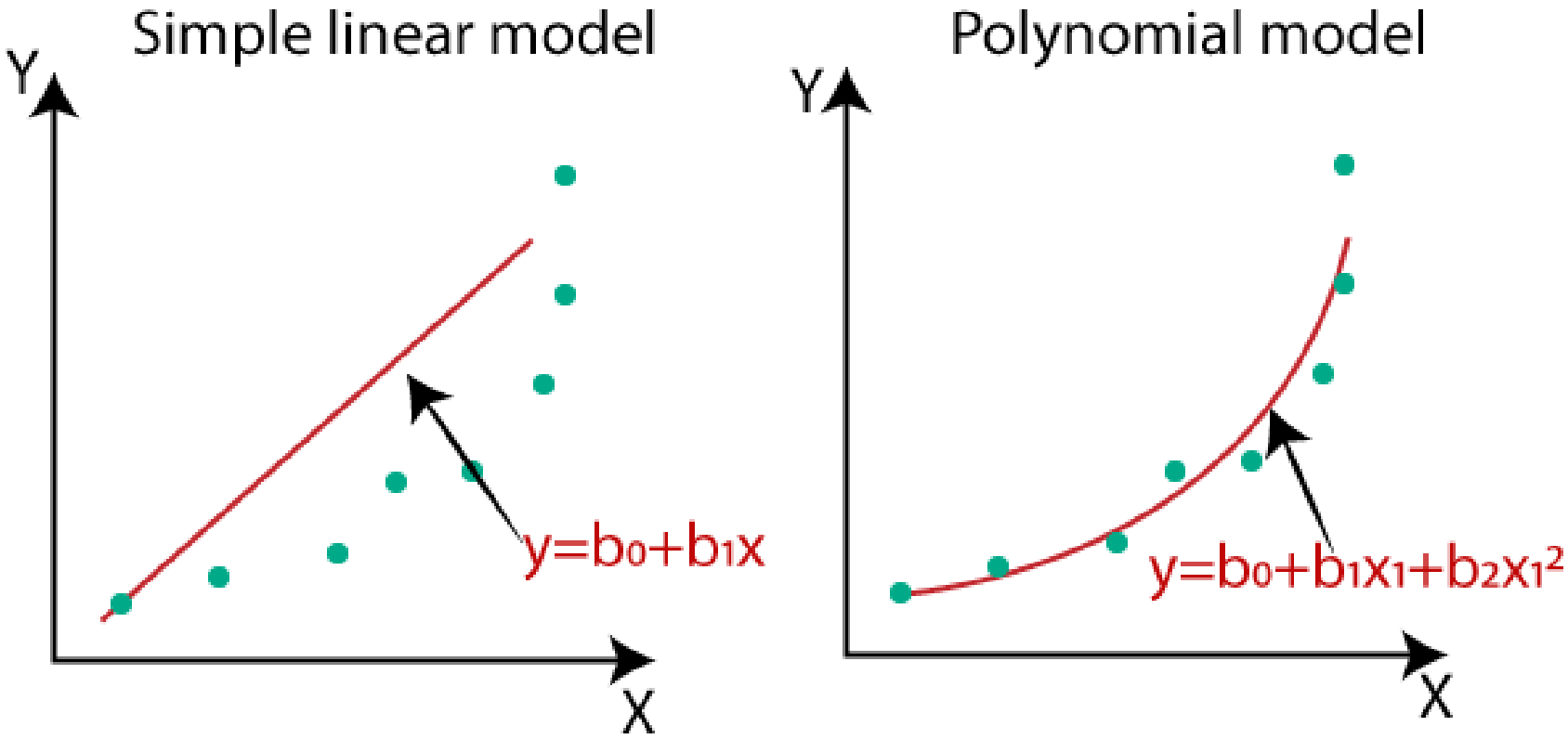
Linear Regression Techniques: Stepwise Regression

Sequential backward selection process

Backward elimination, which involves starting with all candidate variables, testing the deletion of each variable using a chosen model fit criterion, deleting the variable (if any) whose loss gives the most statistically insignificant deterioration of the model fit, and repeating this process until no further variables can be deleted without a statistically significant loss of fit.



Nonlinear Regression: Polynomial Regression



Nonlinear Regression: Polynomial Regression

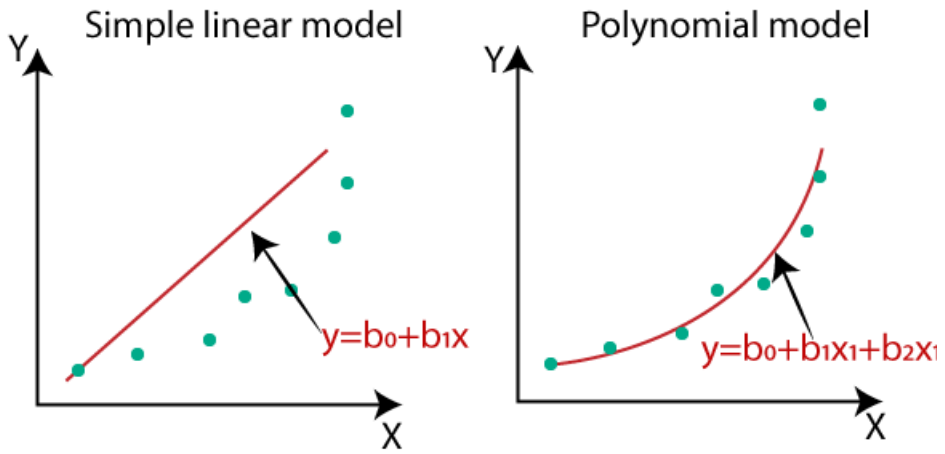
Polynomial regression is a type of regression analysis that models the relationship between the independent variable(s) and the dependent variable as an n-th degree polynomial function. It extends the concept of simple linear regression by introducing higher-order polynomial terms.

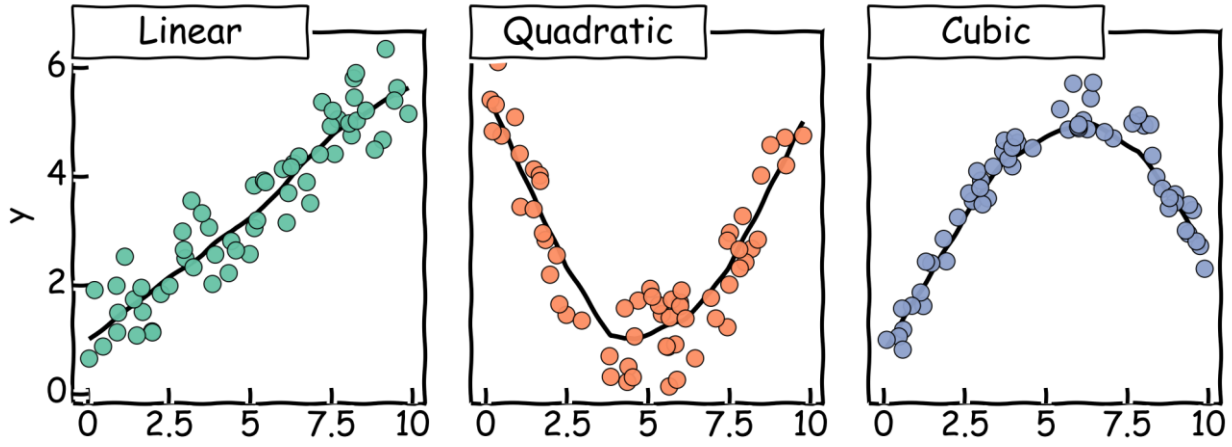
- **Univariate polynomial regression** involves fitting a polynomial function to a single independent variable (or feature) and a single dependent variable. The model assumes that the relationship between the independent and dependent variables can be approximated by a polynomial equation of a specified degree.

$$y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \ldots + \beta_n X^n$$

- **Multivariate polynomial regression** extends the concept of polynomial regression to multiple independent variables. It allows for modeling the relationship between a dependent variable and multiple independent variables using a polynomial equation of a specified degree.

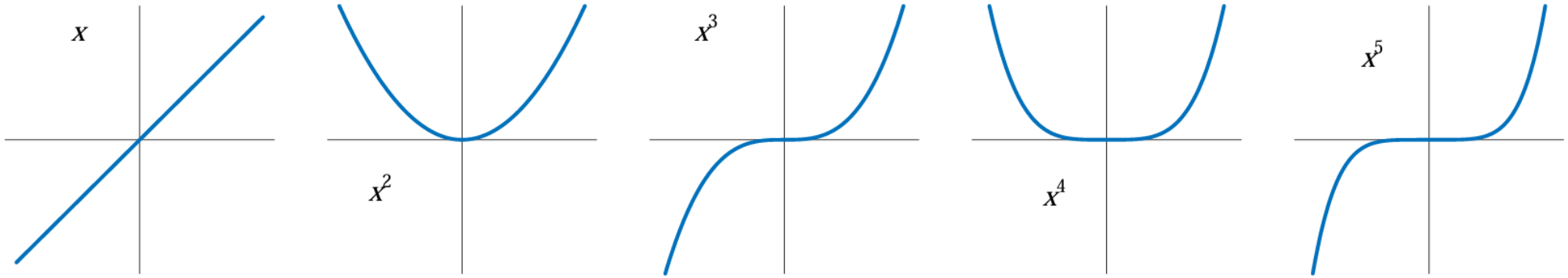
$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p + \beta_{p+1} X_1^2 + \beta_{p+2} X_1 X_2 + \ldots + \beta_{2n+p} X_p^n$$



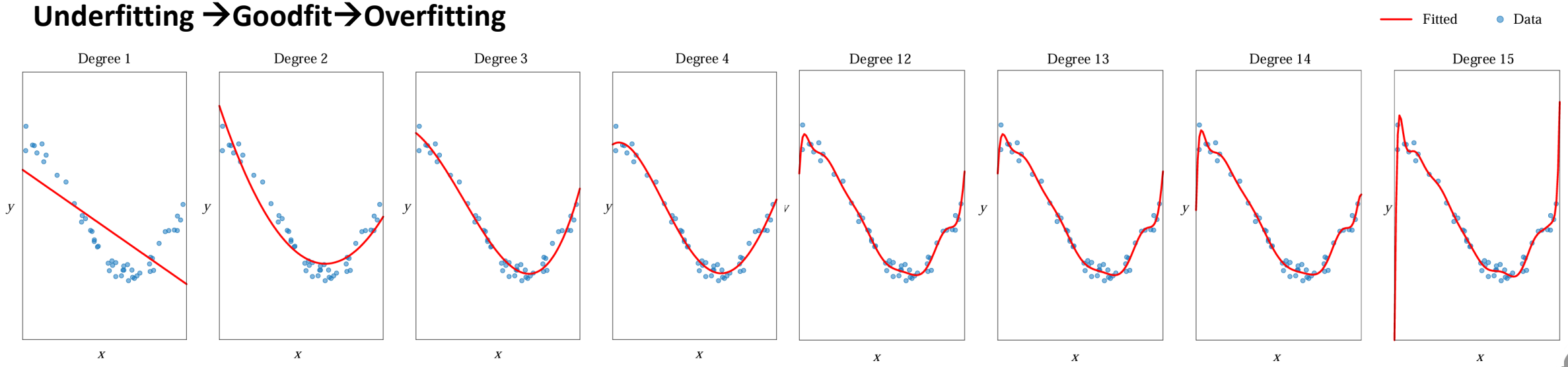


Nonlinear Regression: Univariate Polynomial Regression

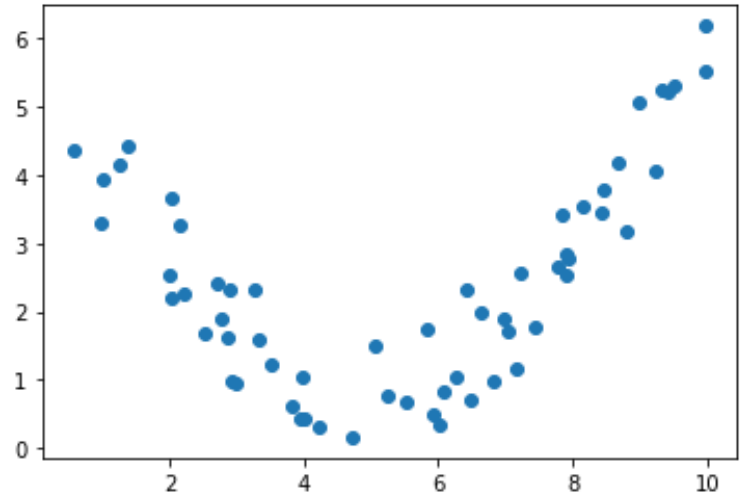
One-variable function of one to five degrees



Underfitting →Goodfit→Overfitting



Nonlinear Regression: Univariate Polynomial Regression




```
X = np.random.random(60)*10      # Input  feature for sample data creation
y = (-X+5)**2/5+np.random.random(len(X))*2   # Target variable

degree = 3  # Degree of polynomial to create polynomial features
X_poly = np.r_[[X**i for i in range(0, degree+1)]].T

df_data=pd.DataFrame(data=np.c_[X_poly,y],columns=["constant","x","x^2","x^3","y"])
selected_features = stepwise_regression(df_data[["constant","x","x^2", "x^3"]], df_data.y)
#Stepwise regression

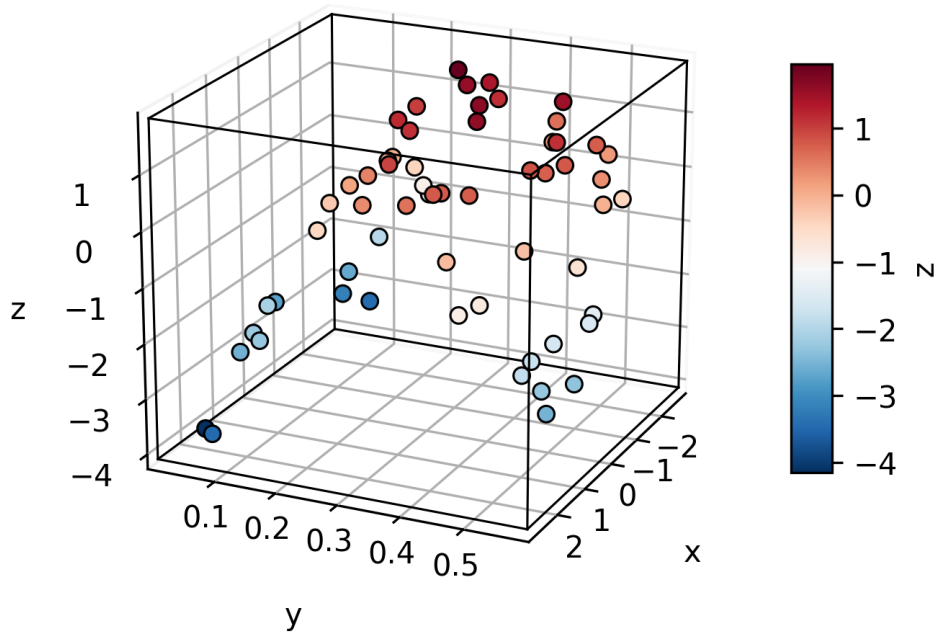
model = sm.OLS(df_data.y, df_data[selected_features])  # Fit the polynomial regression model
results = model.fit()
print(results.summary())  #Print the model summary
```

Nonlinear Regression: Multivariate Polynomial Regression

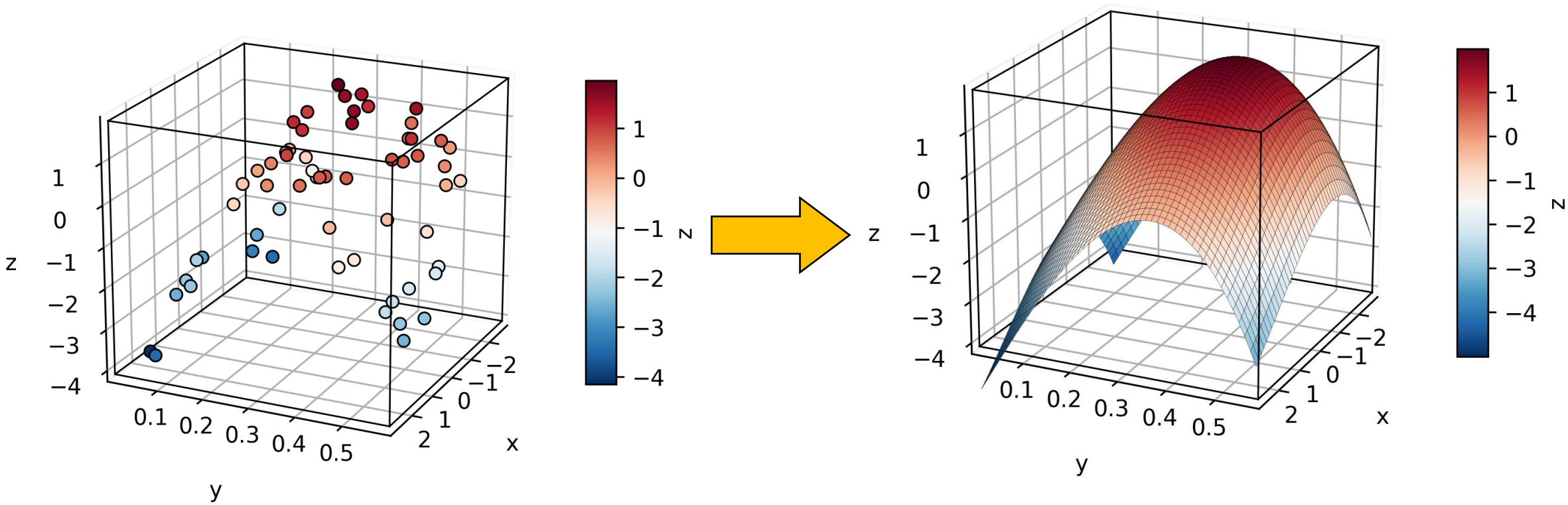
```
from statsmodels.formula.api import ols

x=  (np.random.random(60)-0.5)*6
y=  np.random.random(60)*0.6
z = 0.042*x**3- 0.226*x**2- 0.476*x - 48.434*y**3 + 19.164*y -
3.136+np.random.random(60)*0.5
df=pd.DataFrame(dict(x=x,y=y,z=z))

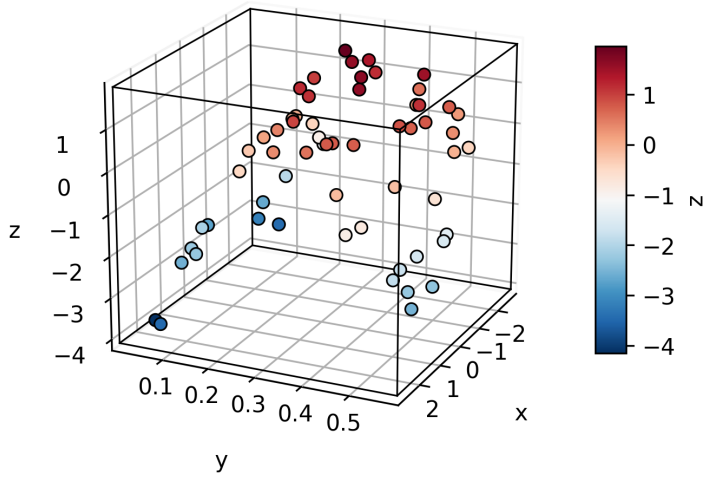
model = ols(formula='z ~y+I(y**3)+x+I(x**2)+I(x**3)',data=df).fit()
print(model.summary())
```

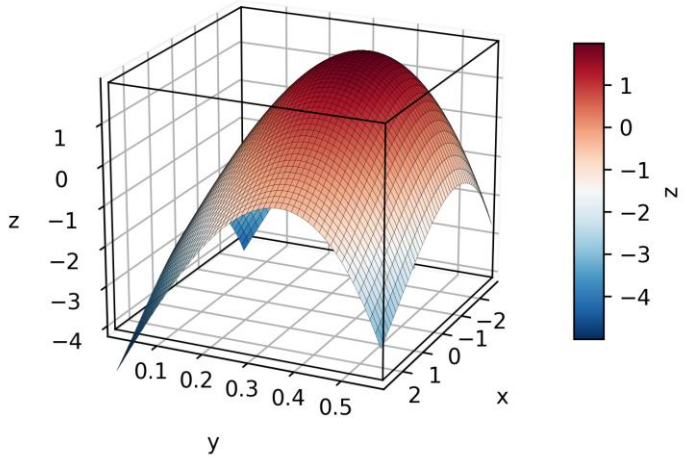


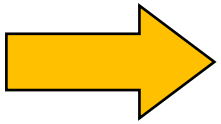
Nonlinear Regression: Multivariate Polynomial Regression

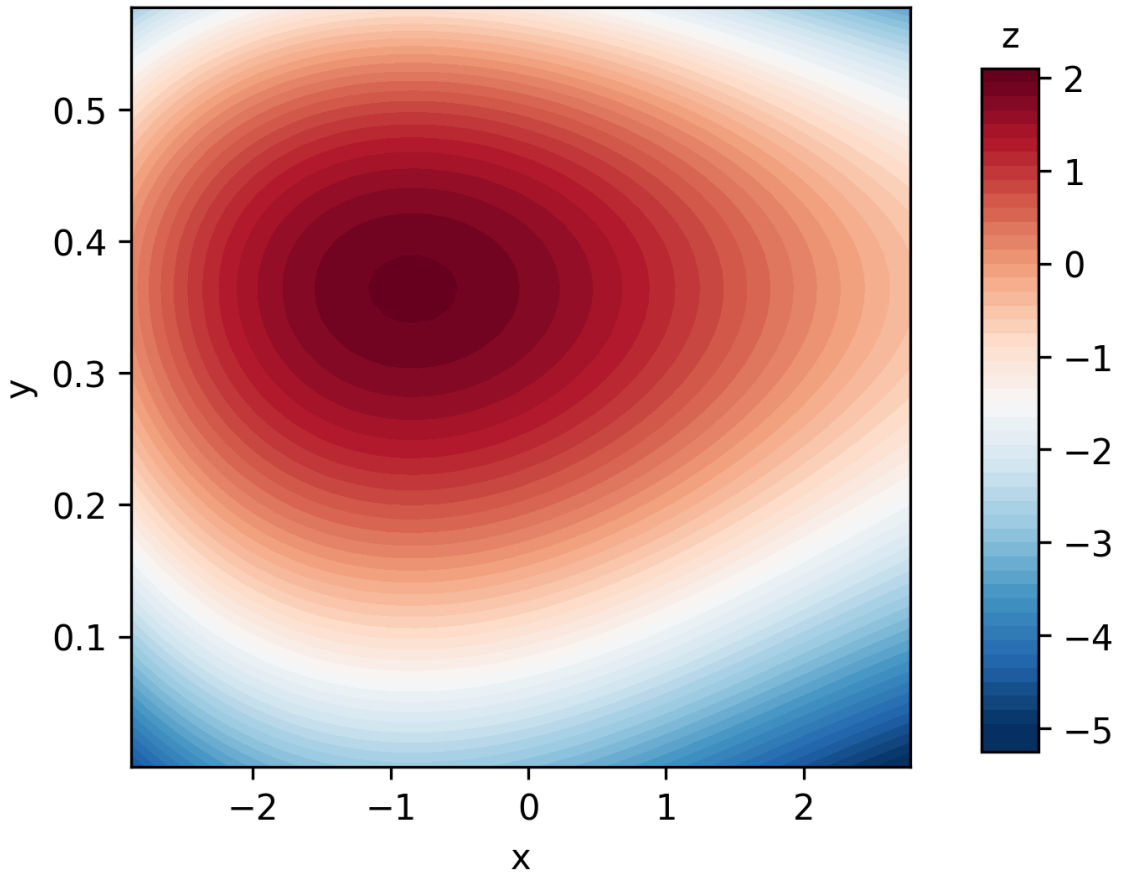


Nonlinear Regression: Multivariate Polynomial Regression







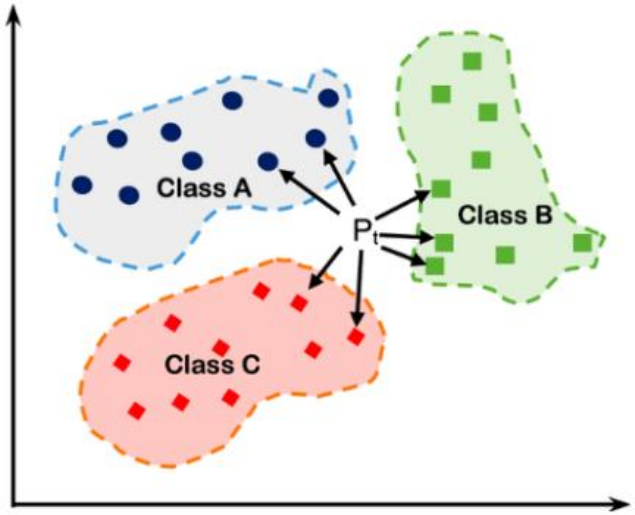


Classification



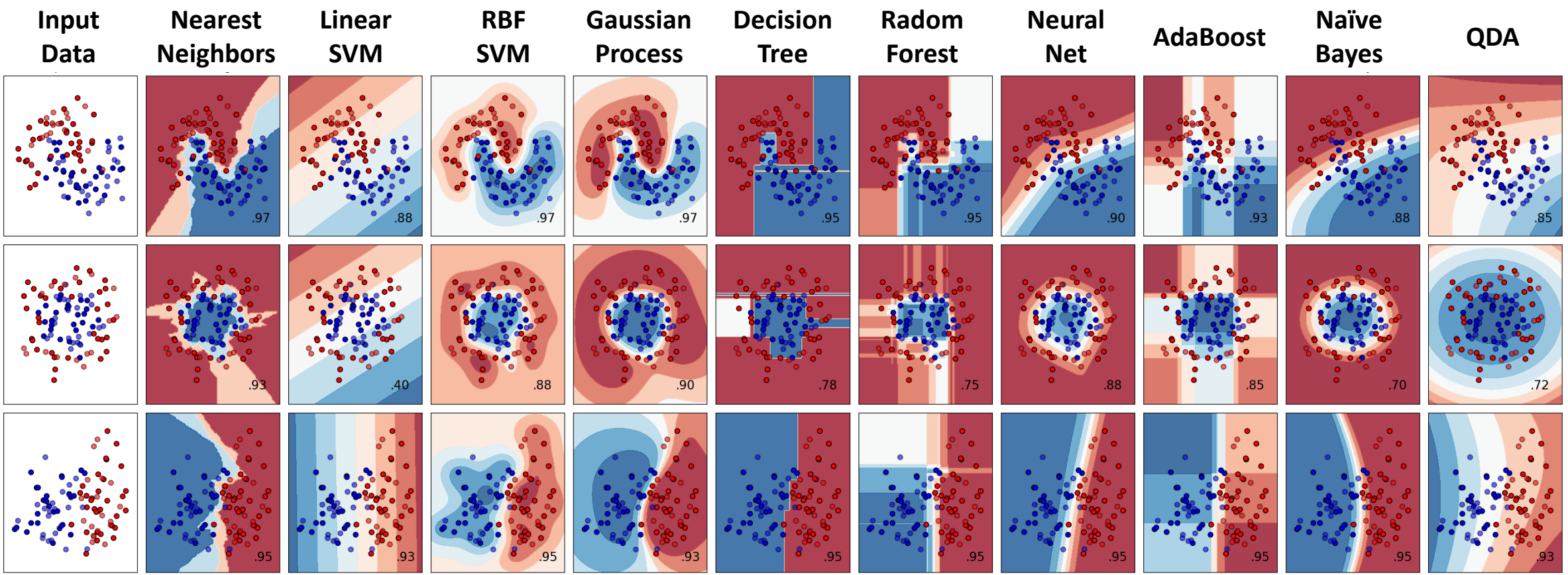
Classification





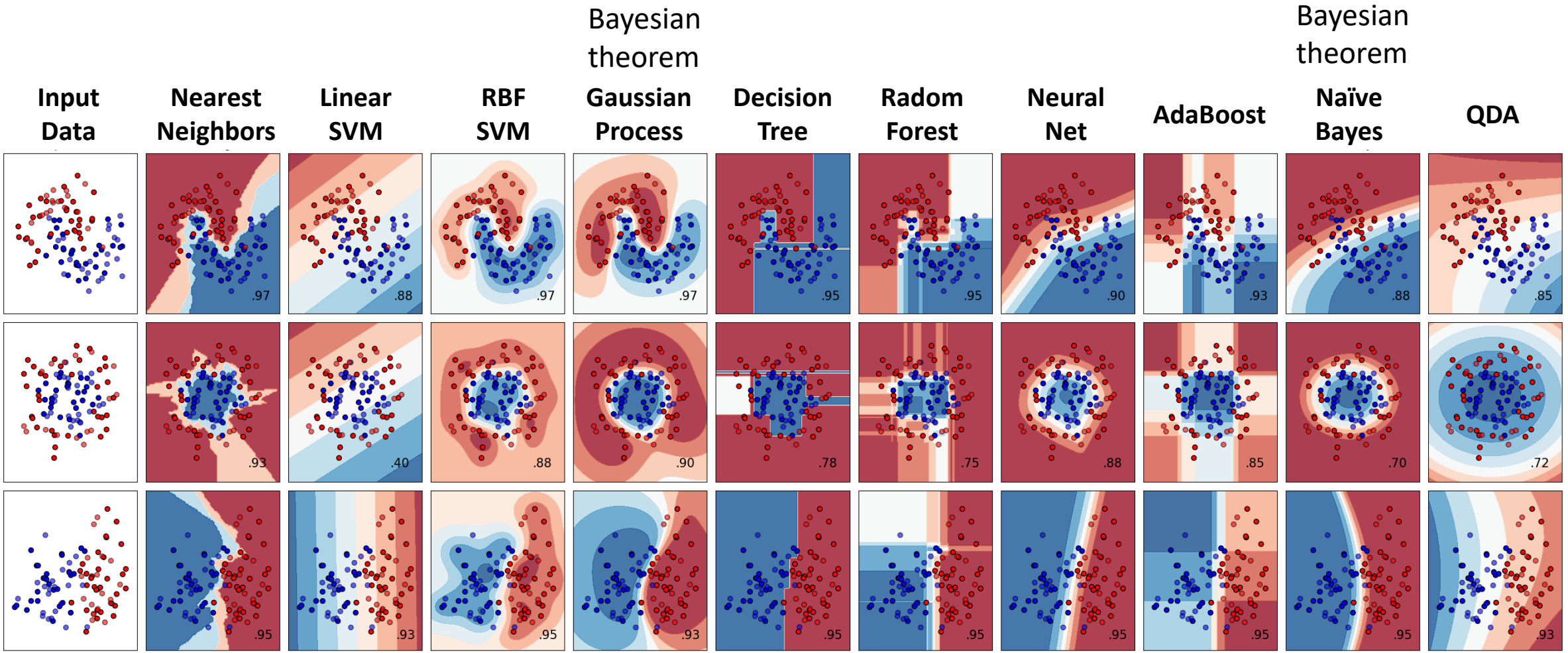
Source: https://medium.com/swlh/k-nearest-neighbor-ca2593d7a3c4

Classifier Comparison



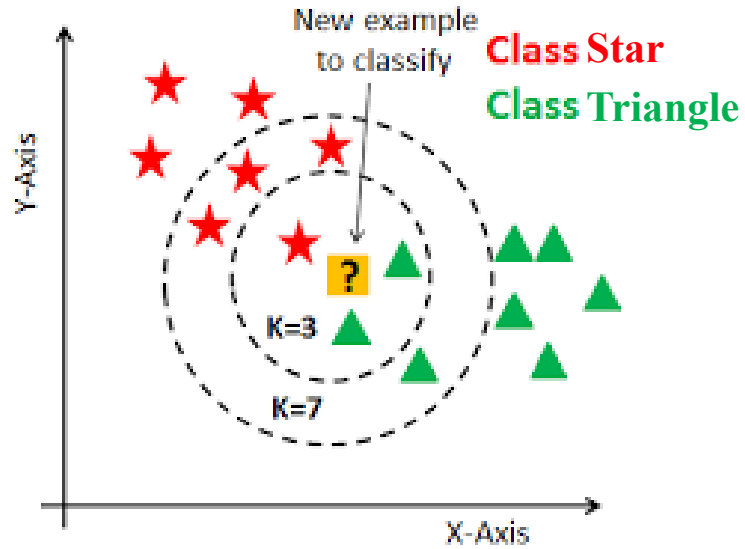
Source: https://scikit-learn.org/stable/

Classifier Comparison



Source: https://scikit-learn.org/stable/

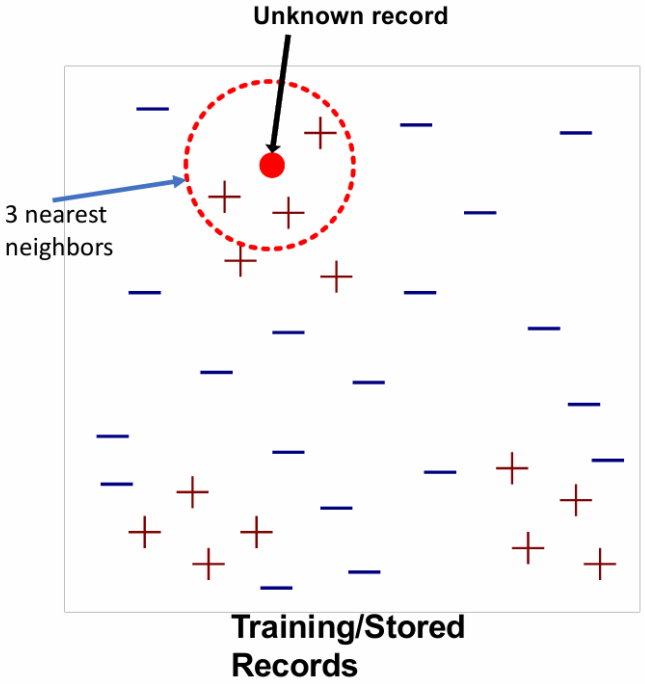
k-nearest neighbors algorithm (KNN)



k-nearest neighbors algorithm

The k-Nearest Neighbors (k-NN) algorithm is a simple, instance-based learning algorithm used for classification and regression tasks in machine learning.

- Instance-Based Learning: k-NN is an instance-based learning algorithm, meaning it memorizes the labeled instances in the training dataset to make predictions.
- Classification and Regression: k-NN can be used for both classification and regression tasks.
- Non-Parametric: It is considered a non-parametric and lazy learning algorithm because it does not make explicit assumptions about the form of the mapping function.



Requires three things:

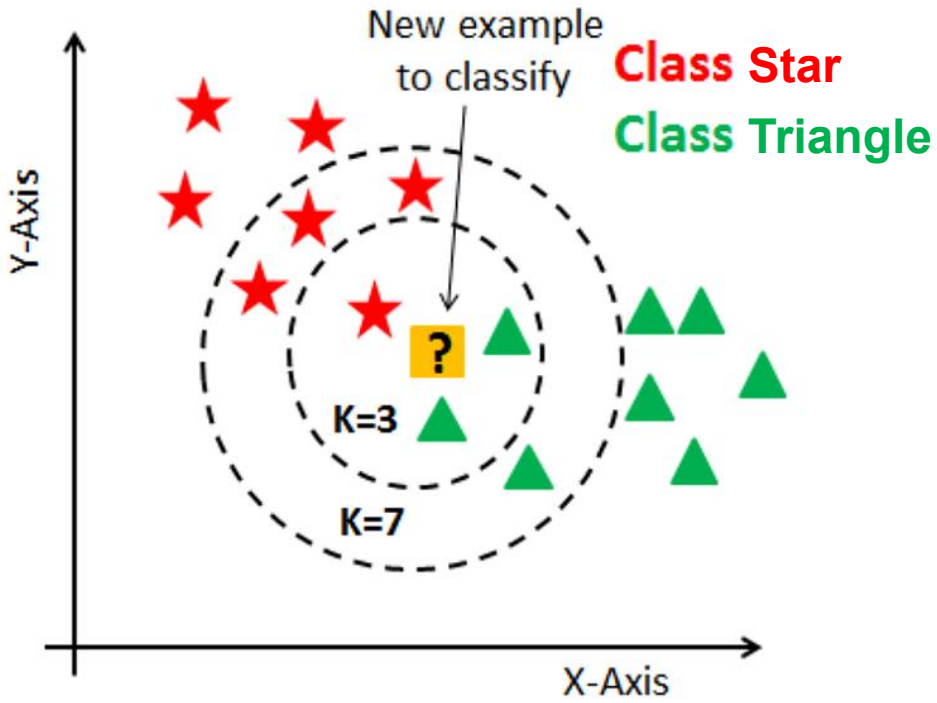
- The set of stored records
- Distance metric to compute distance between records
- The value of k, the number of nearest neighbors to retrieve

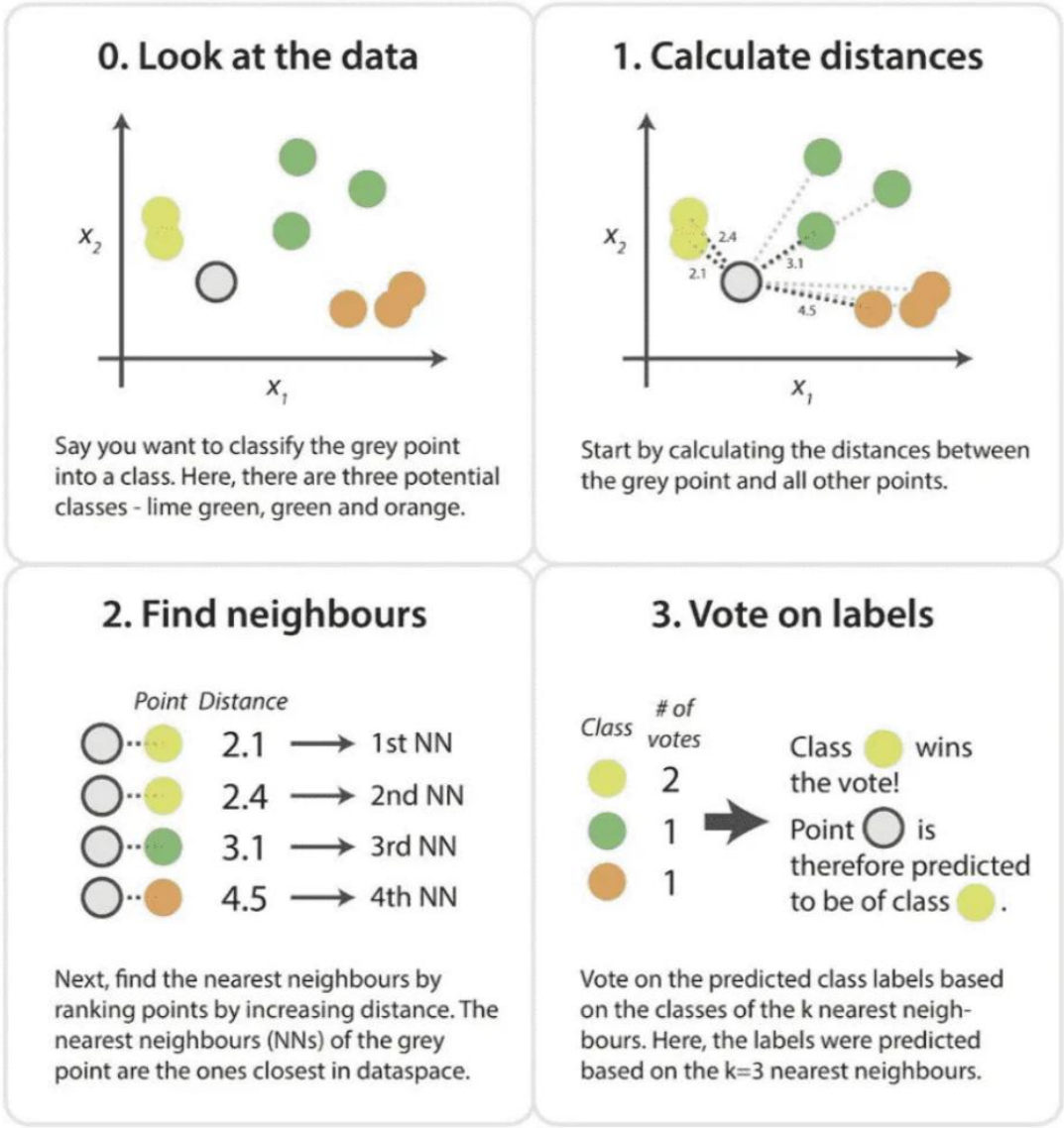
To classify an unknown record:

- Compute distance to other training records– Identify k nearest neighbors
- Use class labels of nearest neighbors to determine the class label of unknown record (e.g., by taking majority vote)



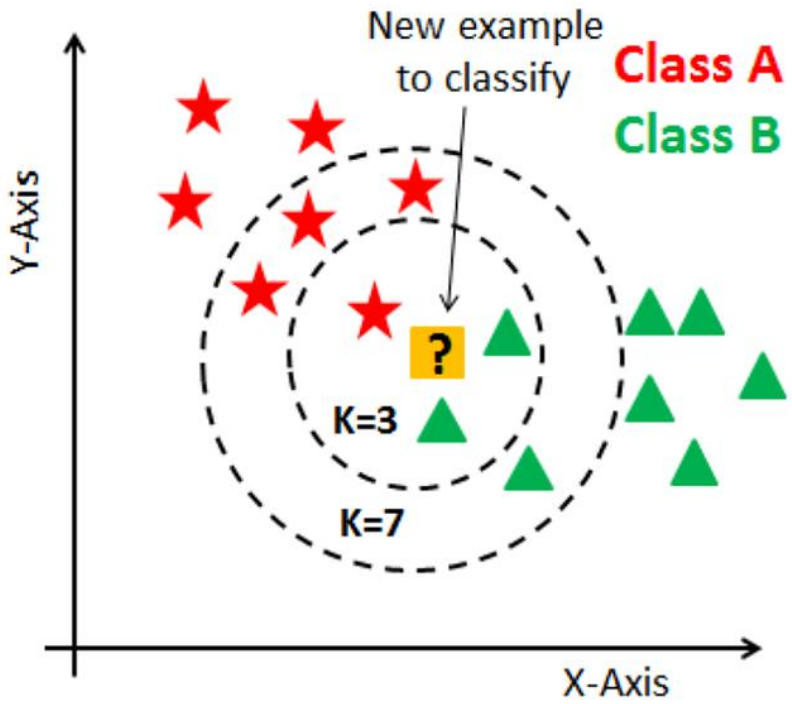
k-nearest neighbors algorithm





k-nearest neighbors algorithm

Choosing k: The choice of the parameter k (number of neighbors) is crucial. A smaller k value makes the model more sensitive to noise, while a larger k value makes the model smoother but might overlook local patterns.



Source: Medium

kNN is one of the simplest yet powerful supervised ML algorithms. It is widely used for classification problems as well as can be used for regression problems.

The data-point is classified on the basis of its k Nearest Neighbors, followed by the majority vote of those nearest neighbors; a query point is assigned the data class which has the most representatives within the nearest neighbors of the point.

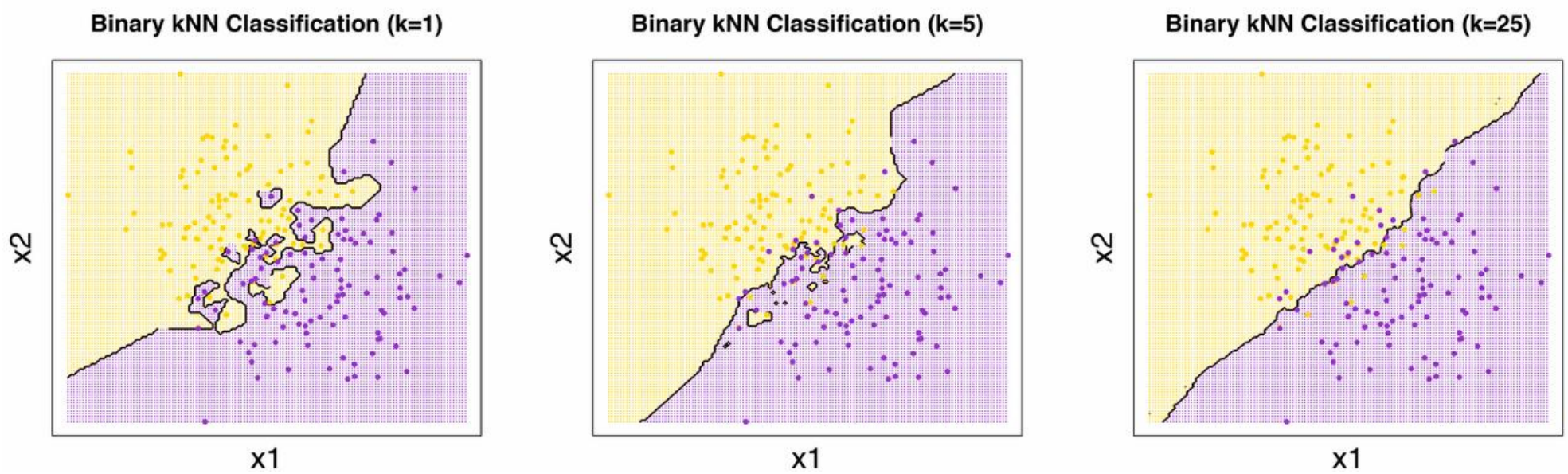
- ☐ When K=1, what is the class of new example?
- ☐ When K=3, what is the class of new example?
- ☐ When K=7, what is the class of new example?

k-nearest neighbors algorithm

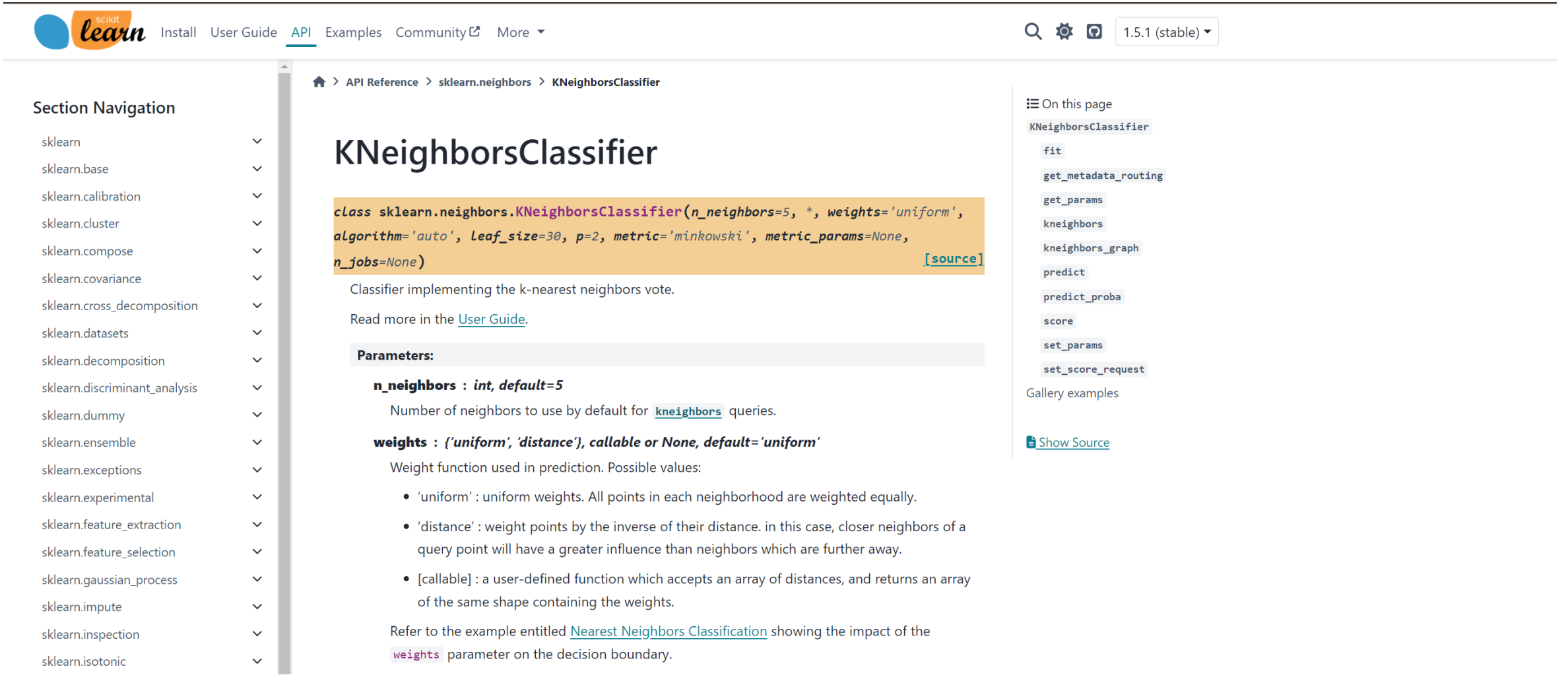
1. **Training**: The algorithm memorizes the feature vectors and their corresponding class labels from the training dataset.
2. **Prediction**:

- For a new input data point, the algorithm calculates the distance (typically Euclidean distance) to all other points in the training set.
- It then selects the k points (neighbors) in the training set that are closest to the new data point.
- For classification, it predicts the class label by **majority** voting among the k-nearest neighbors.
- For regression, it predicts the output by **averaging** the target values of the k-nearest neighbors.

K is a hyperparameter controlling the model complexity



Python-k-nearest neighbors algorithm



scikit learn Install User Guide API Examples Community More 1.5.1 (stable)

Section Navigation

- sklearn
- sklearn.base
- sklearn.calibration
- sklearn.cluster
- sklearn.compose
- sklearn.covariance
- sklearn.cross_decomposition
- sklearn.datasets
- sklearn.decomposition
- sklearn.discriminant_analysis
- sklearn.dummy
- sklearn.ensemble
- sklearn.exceptions
- sklearn.experimental
- sklearn.feature_extraction
- sklearn.feature_selection
- sklearn.gaussian_process
- sklearn.impute
- sklearn.inspection
- sklearn.isotonic

API Reference > sklearn.neighbors > KNeighborsClassifier

KNeighborsClassifier

class sklearn.neighbors.KNeighborsClassifier(*n_neighbors=5, *, weights='uniform', algorithm='auto', leaf_size=30, p=2, metric='minkowski', metric_params=None, n_jobs=None*) [source]

Classifier implementing the k-nearest neighbors vote.

Read more in the User Guide.

Parameters:

n_neighbors : *int, default=5*

Number of neighbors to use by default for `kneighbors` queries.

weights : *{'uniform', 'distance'}, callable or None, default='uniform'*

Weight function used in prediction. Possible values:

- 'uniform' : uniform weights. All points in each neighborhood are weighted equally.
- 'distance' : weight points by the inverse of their distance. in this case, closer neighbors of a query point will have a greater influence than neighbors which are further away.
- [callable] : a user-defined function which accepts an array of distances, and returns an array of the same shape containing the weights.

Refer to the example entitled Nearest Neighbors Classification showing the impact of the `weights` parameter on the decision boundary.

On this page

- KNeighborsClassifier
  - fit
  - get_metadata_routing
  - get_params
  - kneighbors
  - kneighbors_graph
  - predict
  - predict_proba
  - score
  - set_params
  - set_score_request

Gallery examples

Show Source

Source: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html#sklearn.neighbors.KNeighborsClassifier

Python-k-nearest neighbors algorithm

```python
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Create the dataset with 10 samples and 2 features
X = np.array([[0, 0], [1, 1], [2, 2], [3, 3], [4, 4],
              [1, 0], [0, 1], [2, 1], [1, 2], [3, 1]])
y = np.array([0, 0, 1, 1, 1, 0, 0, 1, 0, 1])

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and fit the KNN classifier
neigh = KNeighborsClassifier(n_neighbors=3)
neigh.fit(X_train, y_train)

# Make predictions
prediction = neigh.predict([[1.1, 1.1]])
prediction_proba = neigh.predict_proba([[0.9, 0.9]])

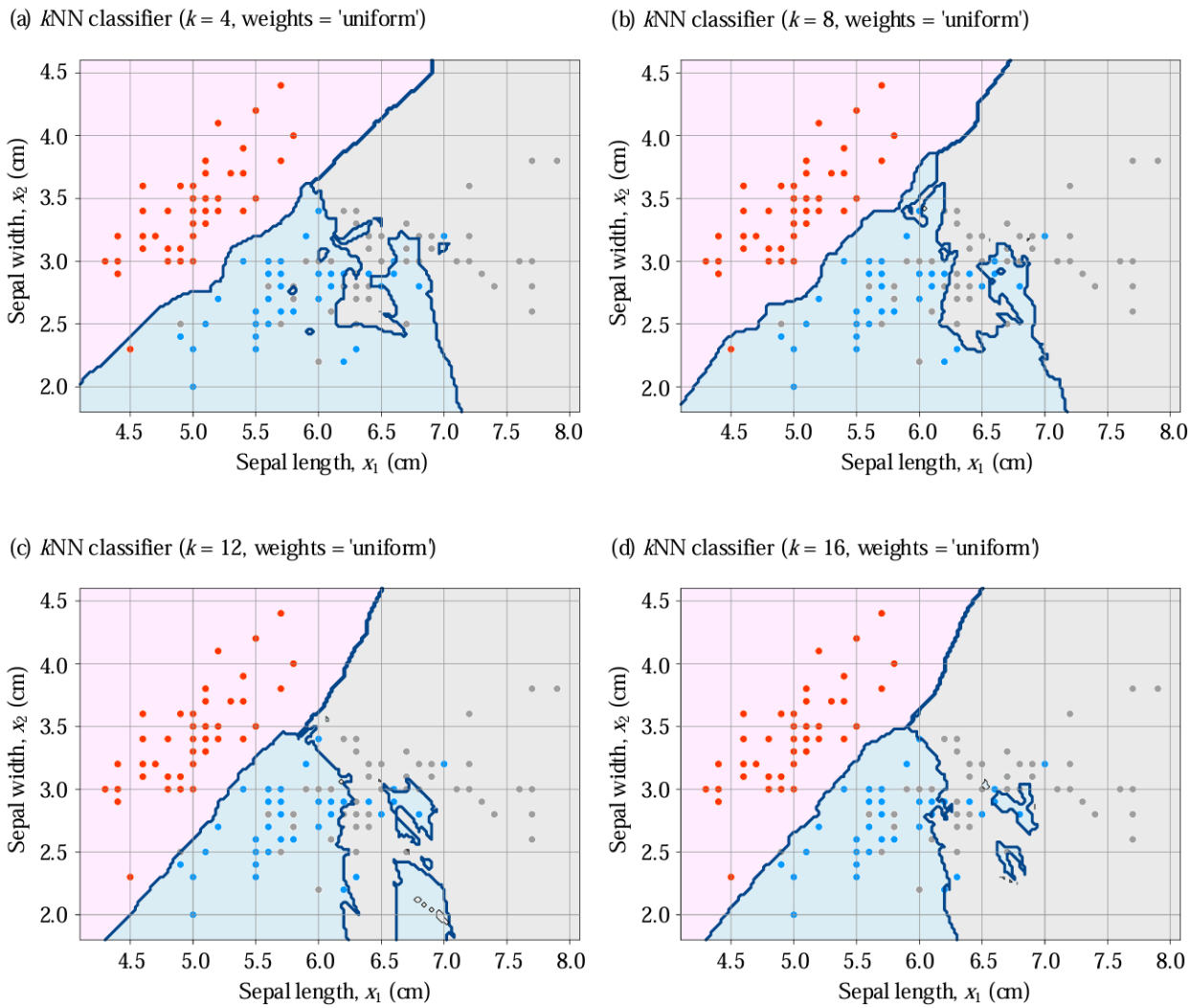
# Print the results
print(f"Predicted class for input [[1.1, 1.1]]: {prediction}")
print(f"Predicted probabilities for input [[0.9, 0.9]]: {prediction_proba}")
```

Source: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html#sklearn.neighbors.KNeighborsClassifier



k-nearest neighbors algorithm

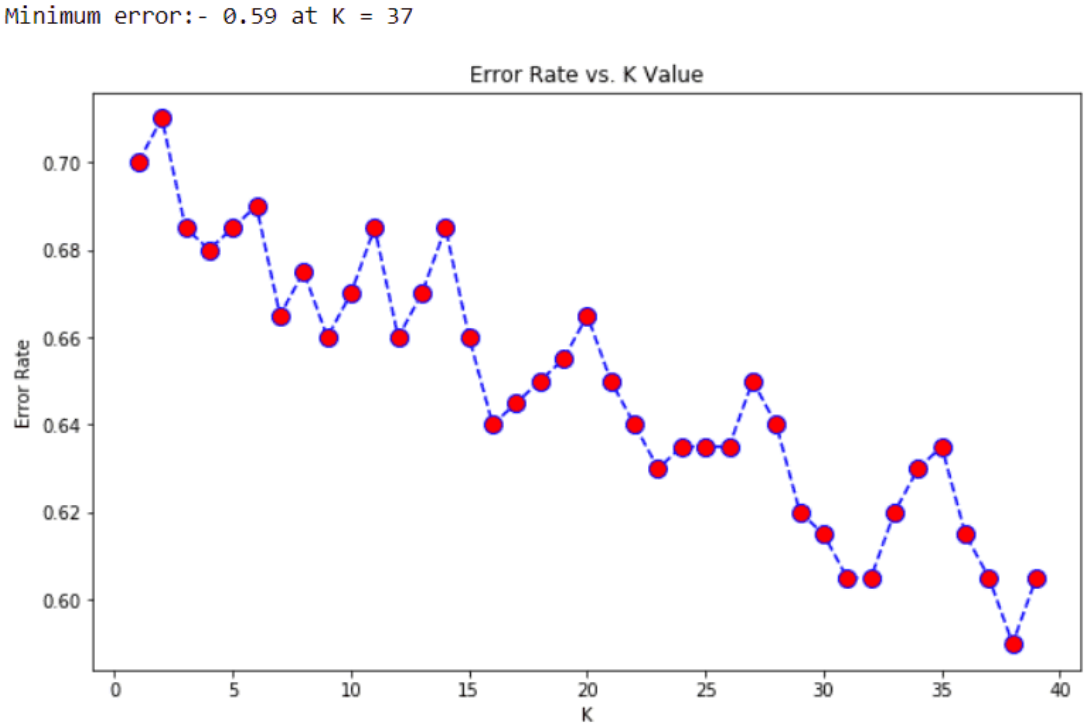
- Although selecting a smaller k value in the k-NN algorithm can accurately capture the classification patterns in the training data, the disadvantage is evident - it is susceptible to noise interference.
- As k gradually increases, the influence of local noisy samples on the boundary diminishes, and the shape of the boundary tends to become smoother.



k-nearest neighbors algorithm

How to select the optimal K value?

1. There are no pre-defined statistical methods to find the most favorable value of K.
2. Initialize a random K value and start computing.
3. Choosing a small value of K leads to unstable decision boundaries.
4. The substantial K value is better for classification as it leads to smoothening the decision boundaries.
5. Derive a plot between error rate and K denoting values in a defined range. Then choose the K value as having a minimum error rate.



k-nearest neighbors algorithm

Advantages:

1. **No Training Period**- KNN modeling does not include training period as the data itself is a model which will be the reference for future prediction and because of this it is very time efficient in term of improvising for a random modeling on the available data.
2. **Easy Implementation**- KNN is very easy to implement as the only thing to be calculated is the distance between different points on the basis of data of different features and this distance can easily be calculated using distance formula such as- Euclidian or Manhattan
3. As there is no training period thus new data can be added at any time since it wont affect the model.

Disadvantages:

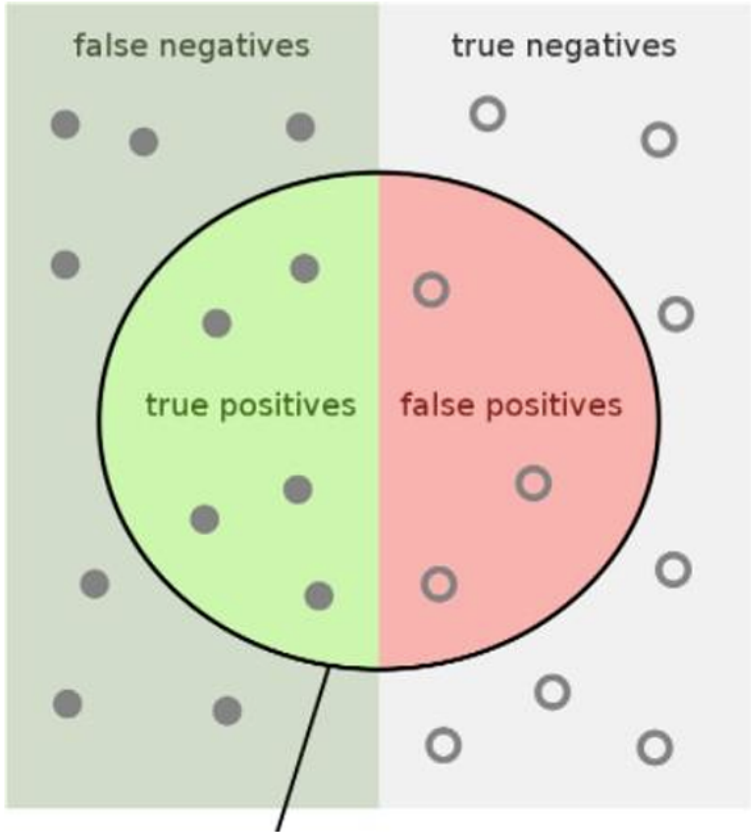
1. **Does not work well with large dataset** as calculating distances between each data instance would be very costly.
2. **Does not work well with high dimensionality** as this will complicate the distance calculating process to calculate distance for each dimension.
3. **Sensitive to noisy and missing data**
4. **Feature Scaling-** Data in all the dimension should be scaled (normalized and standardized) properly .



Classification: Evaluation Metrics

Confusion Matrix

- It is important to know how the model make wrong prediction
- In **binary classification**, confusion matrix is a common tool to analyze the predictions



Classification: Evaluation visualization methods

Confusion Matrix

Evaluation Metrics

- Precision(PRE) & Recall Rate(REC)

$$PRE = \frac{TP}{TP + FP}, \quad \text{(the higher, the better)}$$

$$REC = \frac{TP}{TP + FN} = TPR. \quad \text{(the higher, the better)}$$

- F-1 Score

$$F_1 = 2\frac{(PRE * REC)}{PRE + REC}, \quad \text{(the higher, the better)}$$

		Predicted class: P'	Predicted class: N'
Actual Class	P	True Positives (TP)	False Negatives (FN)
	N	False Positives (FP)	True Negatives (TN)



Classification : Evaluation visualization methods

Confusion Matrix

Evaluation Metrics

- Precision(PRE) & Recall Rate(REC)

$$PRE = \frac{TP}{TP + FP}, \quad \text{(the higher, the better)}$$

$$REC = \frac{TP}{TP + FN} = TPR. \quad \text{(the higher, the better)}$$

- F-1 Score

$$F_1 = 2\frac{(PRE * REC)}{PRE + REC}, \quad \text{(the higher, the better)}$$

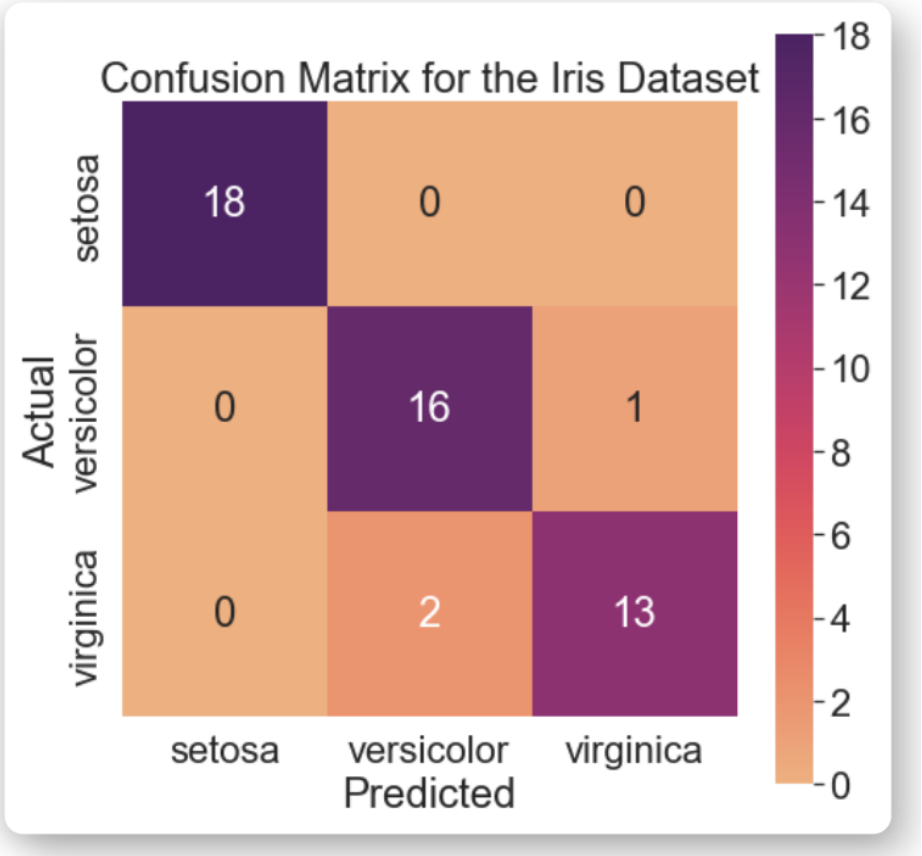
		Predicted class	
		P'	N'
Actual Class	P	True Positives (TP)	False Negatives (FN)
	N	False Positives (FP)	True Negatives (TN)



Confusion matrix in multi-class classification

The diagonal elements represent the number of points for which the predicted label is equal to the true label, while off-diagonal elements are those that are mislabeled by the classifier.

The higher the diagonal values of the confusion matrix the better, indicating many correct predictions.



Source: https://scottminer.rbind.io/post/multi-class-confusion-matrix/
https://scikit-learn.org/stable/auto_examples/model_selection/plot_confusion_matrix.html#confusion-matrix

Classification : Evaluation visualization methods

ROC Curve

- ROC curve analyze the performance for **every threshold in soft classifiers**
- In X-axis $$FPR = \frac{FP}{FP + TN}$$
- In Y-axis $$TPR = \frac{TP}{TP + FN}$$

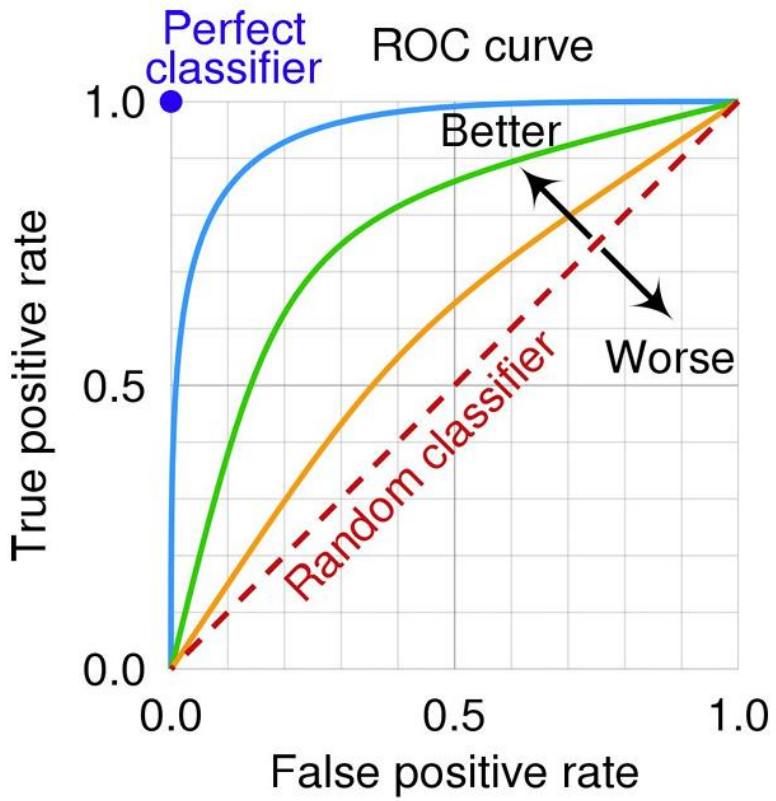
1	
1	
0.87	θ
0.64	⇓
⋮	
−0.88	
−0.93	
−1	



Classification : Evaluation visualization methods

ROC Curve

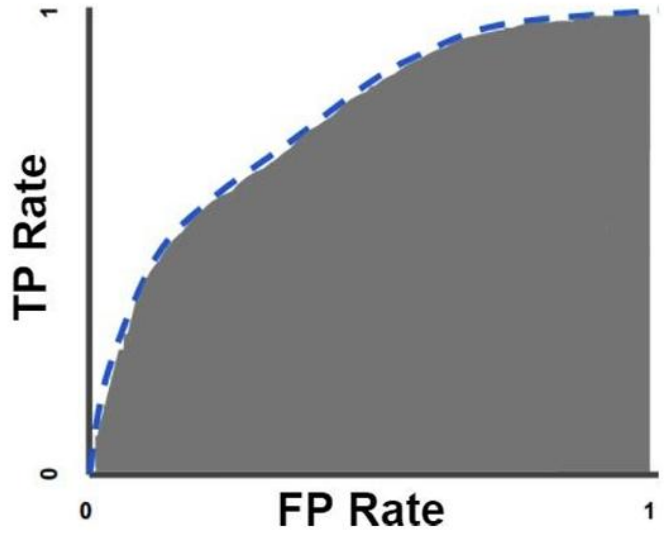
- ROC curve analyze the performance for **every threshold in soft classifiers**
- In X-axis $$FPR = \frac{FP}{FP + TN}$$
- In Y-axis $$TPR = \frac{TP}{TP + FN}$$

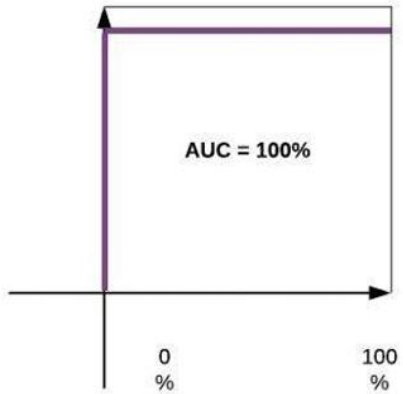


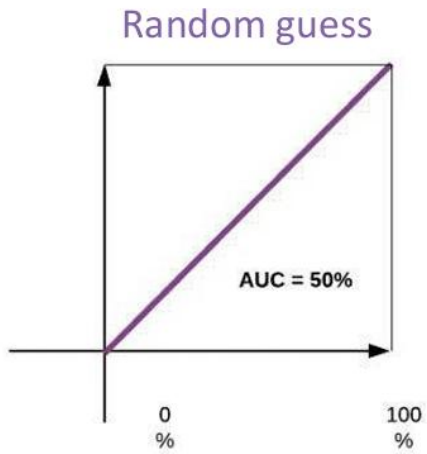
Classification : Evaluation Metrics

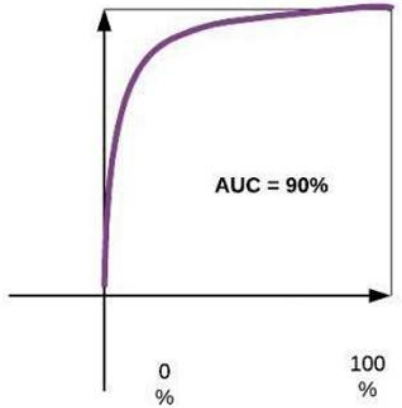
AUC

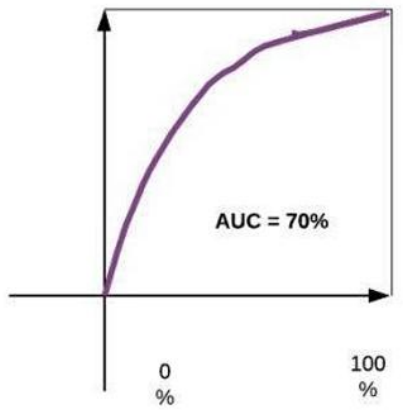
- AUC (Area Under the ROC Curve)
  - ROC can be quantified using AUC











Python-Classification Evaluation Metrics

confusion_matrix

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html

sklearn.metrics.confusion_matrix(*y_true*, *y_pred*, ***, *labels=None*, *sample_weight=None*, *normalize=None*) [source]

Compute confusion matrix to evaluate the accuracy of a classification.

By definition a confusion matrix C is such that $C_{i,j}$ is equal to the number of observations known to be in group i and predicted to be in group j.

Thus in binary classification, the count of true negatives is $C_{0,0}$, false negatives is $C_{1,0}$, true positives is $C_{1,1}$ and false positives is $C_{0,1}$.

Read more in the User Guide.

Parameters:

y_true : *array-like of shape (n_samples,)*

Ground truth (correct) target values.

y_pred : *array-like of shape (n_samples,)*

Estimated targets as returned by a classifier.

labels : *array-like of shape (n_classes), default=None*

List of labels to index the matrix. This may be used to reorder or select a subset of labels. If `None` is given, those that appear at least once in `y_true` or `y_pred` are used in sorted order.

sample_weight : *array-like of shape (n_samples,), default=None*

Sample weights.

roc_curve

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_curve.html#sklearn.metrics.roc_curve

sklearn.metrics.roc_curve(*y_true*, *y_score*, ***, *pos_label=None*, *sample_weight=None*, *drop_intermediate=True*) [source]

Compute Receiver operating characteristic (ROC).

Note: this implementation is restricted to the binary classification task.

Read more in the User Guide.

Parameters:

y_true : *array-like of shape (n_samples,)*

True binary labels. If labels are not either {-1, 1} or {0, 1}, then pos_label should be explicitly given.

y_score : *array-like of shape (n_samples,)*

Target scores, can either be probability estimates of the positive class, confidence values, or non-thresholded measure of decisions (as returned by "decision_function" on some classifiers). For decision_function scores, values greater than or equal to zero should indicate the positive class.

pos_label : *int, float, bool or str, default=None*

The label of the positive class. When `pos_label=None`, if `y_true` is in {-1, 1} or {0, 1}, `pos_label` is set to 1, otherwise an error will be raised.

sample_weight : *array-like of shape (n_samples,), default=None*

Sample weights.

drop_intermediate : *bool, default=True*

Whether to drop some suboptimal thresholds which would not appear on a plotted ROC curve. This is useful in order to create lighter ROC curves.



Python-Classification Evaluation Metrics

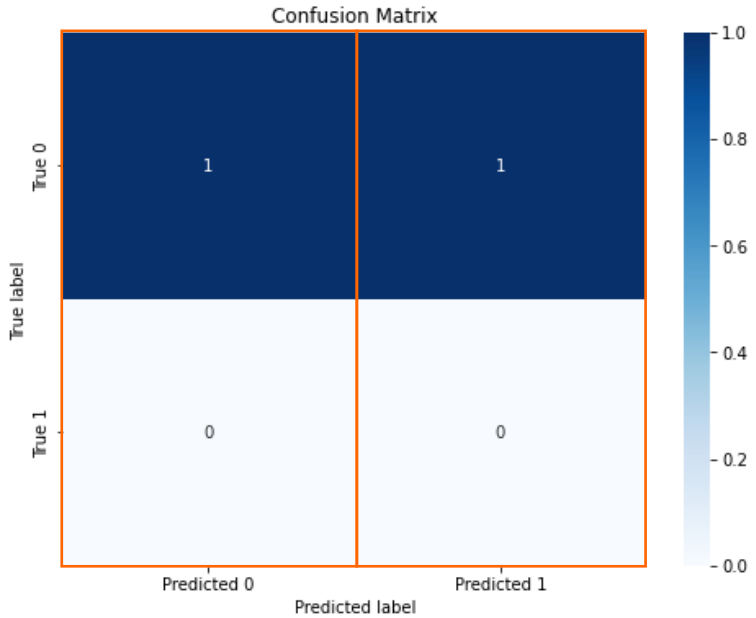
```python
# Make predictions
y_pred = neigh.predict(X_test)
y_pred_proba = neigh.predict_proba(X_test)[:, 1]  # Get probabilities for the positive class

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Plotting the confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['True 0', 'True 1'])
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.title('Confusion Matrix')
plt.show()

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
roc_auc = auc(fpr, tpr)

# Plot ROC Curve
plt.figure()
plt.plot(fpr, tpr, color='blue', label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='red', linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.0])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc='lower right')
plt.show()
```



Thanks

